Phase 2 — Technical Documentation

ESL Orchestrator

11.05.2026
João Pires - Devoteam
Sonae MC

Contents

1. Introduction

1.1. Purpose

This document provides the technical documentation for Phase 2 of the ESL Orchestrator platform at Sonae MC. Building on the data synchronisation pipeline established in Phase 1, Phase 2 adds Change Data Capture (CDC) — the ability to detect when entity records are created, modified, or deleted during each pipeline execution and publish those changes as events to a Solace message broker.

Phase 2 delivers three new components and modifies one existing component:

Component	Role
esl-common (new)	Shared Go library containing CDC event types and entity key definitions, imported by both the Data Pipeline and the Event Publisher
Database — event outbox (new migrations)	Transactional outbox table that stores detected change events atomically alongside data upserts
Data Pipeline — CDC (modified)	Pre-fetch and in-Go classification logic added to the existing sink, writing change events to the outbox within the same database transaction as upserts
Event Publisher (new)	Long-running service that polls the outbox table and publishes events to Solace with guaranteed delivery

1.2. Scope

This document covers the full Phase 2 scope: architecture, database changes, Data Pipeline CDC logic, Event Publisher service, deployment, and operations.

All three CDC event types are covered: CREATED, UPDATED, and DELETED. DELETED detection applies to products and labels only (stores and access points have no deletion lifecycle in Vusion).

1.3. Audience

This document is intended for the same audience as Phase 1:

- Technical stakeholders who need to understand the CDC architecture and its integration with the existing platform
- Developers who will extend or maintain the CDC pipeline and Event Publisher
- Operations teams who will deploy, configure, and monitor the new components
- Project managers and product owners who need visibility into the platform's new capabilities

1.4. How to Read This Document

The document is structured in the following order:

1. System Architecture — Updated high-level view showing how CDC fits into the existing platform
2. Database — Event outbox table, indexes, and retention
3. Data Pipeline — CDC detection logic in the sink
4. Event Publisher — Outbox polling and Solace publishing
5. Deployment & Infrastructure — Helm chart changes for the new components
6. Operations & Maintenance — Monitoring, outbox management, and troubleshooting

1.5. Glossary

Terms introduced in Phase 1 still apply. The following terms are new in Phase 2:

Term	Description
CDC	Change Data Capture — detecting and recording data changes as they occur
Transactional Outbox	Pattern where change events are written to a database table within the same transaction as the data change, guaranteeing atomicity
Event Publisher	Service that polls the outbox table and publishes events to a message broker
Solace	Message broker used for event distribution to downstream consumers
Outbox	The `event_outbox` table that acts as a reliable buffer between change detection and event publishing
CREATED event	Event emitted when a record is inserted for the first time; payload contains the full record snapshot
UPDATED event	Event emitted when an existing record's non-audit fields change; payload contains only the changed fields as `{"field": {"old": X, "new": Y}}` diffs
DELETED event	Event emitted when a record's status transitions to `DELETED`; payload is empty — the published message carries only entity key, `eventId`, and `send_date`
Entity key	Composite business key that uniquely identifies a record (e.g., `retail_chain_id + store_id` for stores)
esl-common	Shared Go library providing CDC types and entity definitions used by multiple components

2. System Architecture

2.1. Overview

Phase 2 extends the ESL Orchestrator with a CDC (Change Data Capture) pipeline that detects data changes during each sync and publishes them as events. The existing Phase 1 architecture — ingest, store, serve — remains unchanged. Phase 2 adds a fourth stage: detect and publish.

The CDC pipeline uses the transactional outbox pattern: change events are written to an outbox table within the same database transaction as the data upserts, eliminating dual-write consistency problems. A separate Event Publisher service polls the outbox and delivers events to Solace.

2.2. Architecture

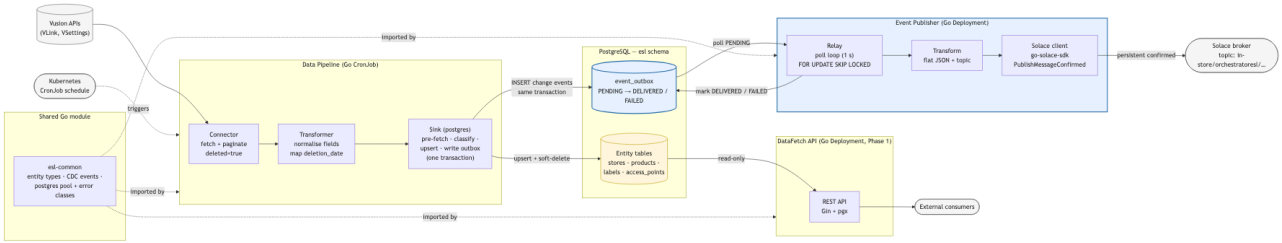


Figure 1: Phase 2 Architecture

2.3. CDC Data Flow

The CDC data flow adds three steps to the existing upsert path:

1. Pre-fetch — Before upserting a batch, the sink queries the current state of the target rows within a database transaction
2. Classify — Each incoming record is compared against the pre-fetched state:
   - Existing `status` already `DELETED` and incoming `status` also `DELETED` → no event (skip fast)
   - Incoming `status` is `DELETED` (transition or initial full sync) → DELETED (payload: empty)
   - Key not found → CREATED (payload: full record snapshot)
   - Key found, fields differ → UPDATED (payload: changed fields as old/new diffs)
   - Key found, all fields identical → no event
3. Outbox write — Detected change events are inserted into `esl.event_outbox` within the same transaction as the upsert batch
4. Publish — The Event Publisher polls undelivered outbox rows, publishes each to Solace, and marks them as delivered

The transactional outbox guarantees that change events are only recorded when the corresponding data changes are committed. If the transaction rolls back, both the upsert and the outbox writes are discarded.

2.4. Component Responsibilities

Component	Type	Technology	Responsibility
Data Pipeline	CronJob	Go, pgx	Fetch data from Vusion APIs, transform, upsert, and detect changes (CDC)
Database	Job (PreSync)	PostgreSQL 18, Flyway 12	Schema management, data storage, event outbox
DataFetch API	Deployment	Go, Gin, pgx	Read-only REST API (unchanged from Phase 1)
Event Publisher	Deployment	Go, pgx, Solace SDK	Poll outbox, publish events to Solace, mark delivered
esl-common	Go library	Go	Shared CDC event types and entity key definitions

2.5. Shared Library: esl-common

The `esl-common` module provides types and definitions shared between the Data Pipeline and Event Publisher, ensuring consistent serialisation and entity key handling.

Module: `github.com/sonaemc-instore/lac1041-instoreorchestrator_esl-common`

2.5.1. Packages

`event` — CDC event types:

```go
type ChangeType string
const (
    ChangeTypeCreated ChangeType = "CREATED"
    ChangeTypeUpdated ChangeType = "UPDATED"
    ChangeTypeDeleted ChangeType = "DELETED"
)

type ChangeEvent struct {
    EventType  ChangeType        `json:"event_type"`
    EntityType string            `json:"entity_type"`
    EntityKey  map[string]string `json:"entity_key"`
    Payload    map[string]any    `json:"payload"`
    OccurredAt time.Time         `json:"occurred_at"`
}
```

`entity` — Entity type enumeration and composite business keys:

```
type EntityType string
const (
    Store       EntityType = "store"
    Product     EntityType = "product"
    Label       EntityType = "label"
    AccessPoint EntityType = "accesspoint"
)

var ConflictKeys = map[EntityType][]string{
    Store:       {"retail_chain_id", "store_id"},
    Product:     {"retail_chain_id", "store_id", "item_id"},
    Label:       {"store_id", "retail_chain_id", "label_id"},
    AccessPoint: {"retail_chain_id", "store_id", "id"},
}
```

These conflict keys serve a dual purpose: they define the `ON CONFLICT` columns for upsert queries and the composite key included in each change event's `entity_key` field.

2.6. Key Design Decisions

2.6.1. Transactional outbox over CDC triggers

Database-level triggers (e.g., `LISTEN/NOTIFY` or logical replication) were considered but rejected in favour of application-level change detection with a transactional outbox. The outbox approach keeps all CDC logic in Go — where it can be tested, feature-flagged, and deployed independently — while guaranteeing atomicity with the upsert.

2.6.2. Pre-fetch and in-Go classification

Detecting changes requires knowing the previous state of each record. Since UPDATED events carry a diff payload (`{"field": {"old": X, "new": Y}}`), pre-fetching the current rows is mandatory. Given that pre-fetch is already needed, classification (CREATED vs UPDATED vs unchanged) is performed in Go at negligible additional cost.

2.6.3. Separate publisher service

The Event Publisher runs as its own Deployment rather than being embedded in the Data Pipeline CronJob. This decouples publish latency from sync execution, allows independent scaling and restarts, and means a Solace outage does not block data synchronisation.

2.6.4. Horizontal scaling of the Event Publisher

The Event Publisher uses `SELECT FOR UPDATE SKIP LOCKED` to claim outbox rows. This naturally supports multiple replicas — PostgreSQL skips rows already locked by other transactions, so each replica auto-

matically gets a distinct set of rows with no coordination needed. The only caveat is event ordering per entity: with multiple replicas, two events for the same entity could be picked up by different replicas and published out of order. If downstream consumers require per-entity ordering, this would need partitioning by entity key (e.g. consistent hashing). For now, a single replica is sufficient.

2.6.5. Feature flag

CDC is gated behind a `cdc.enabled` configuration flag in the sink. When disabled, the pipeline behaves exactly as in Phase 1 — no transaction wrapper, no pre-fetch, no outbox writes, and the VLink connector skips fetching deleted rows.

2.6.6. Per-record outcome reporting

The sink reports the persistence outcome of each record back to the rest of the pipeline through an `OutcomeHandler` callback installed at startup. The handler is invoked exactly once per record before `Close` returns — `nil` on successful persistence, a non-nil error on permanent failure. This contract decouples the sink from the connector's bookkeeping: the connector and metrics layer learn what actually persisted (vs. what was merely emitted to the sink channel), without the sink needing to know which `(store, entity)` tuple a record belongs to.

Two implementation details follow from PostgreSQL's batch semantics:

- Non-CDC: `pool.SendBatch` runs all queued queries in an implicit transaction, so a single failure rolls back even queries whose `br.Exec` reported success. The sink recovers the per-record truth by re-executing every record individually via `pool.Exec` after a batch failure — each runs in its own implicit transaction and gets its true outcome. Records whose data is valid persist; only the genuine violators carry a non-nil error in their outcome.
- CDC: the explicit transaction wrapping the batch + outbox writes makes per-record retries unsafe (they would break upsert/outbox atomicity). Outcomes are uniform: commit-success → all `nil`; any failure → all records carry a `rolledBackErr` wrapping the root cause.

3. Database

3.1. Overview

Phase 2 adds a single new table — `event_outbox` — to the existing `esl` schema. This table implements the transactional outbox pattern: the Data Pipeline inserts change events into the outbox within the same database transaction as entity upserts, and the Event Publisher polls the outbox to deliver events to Solace.

All existing Phase 1 tables remain unchanged.

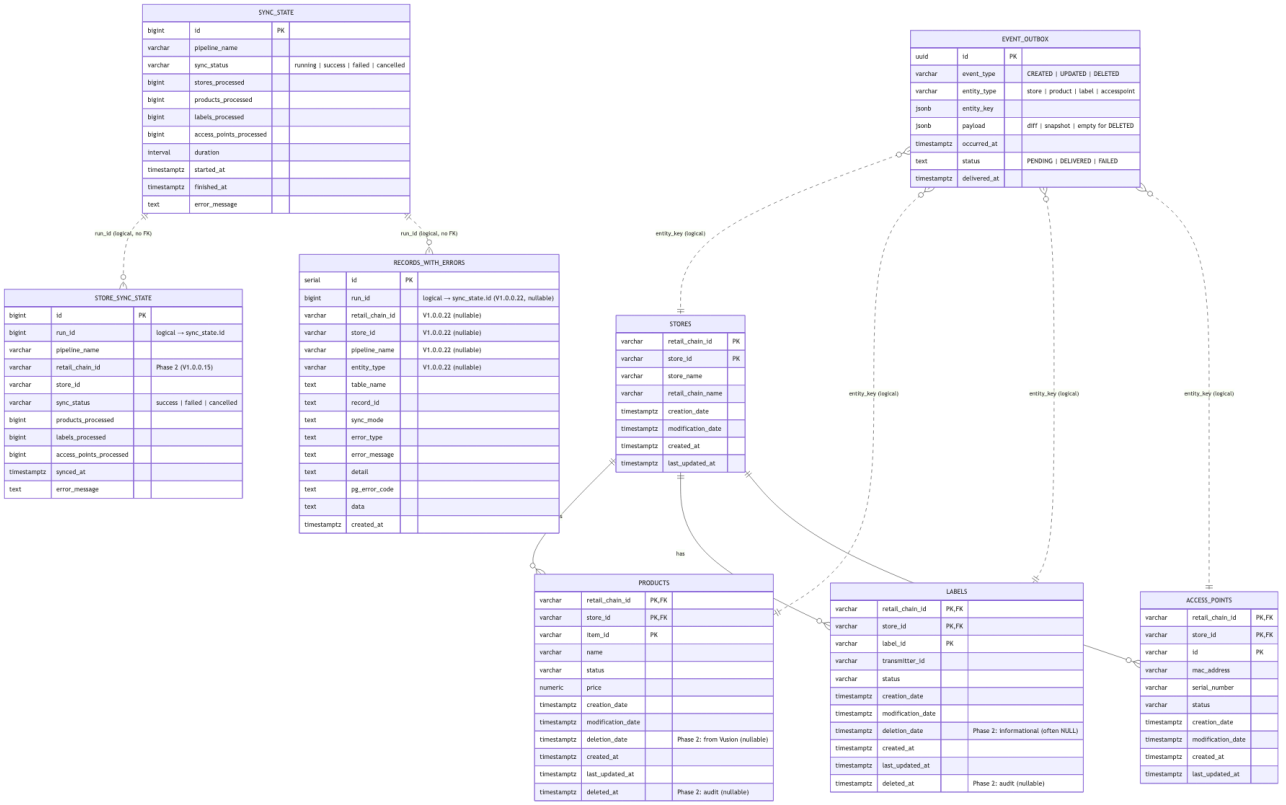


Figure 2: ER Diagram

3.2. Event Outbox Table

3.2.1. event_outbox

Stores CDC events detected during pipeline execution. Each row represents a single CREATED, UPDATED, or DELETED event for one entity record.

Column	Type	Default	Description
`id`	UUID	`gen_random_uuid()`	Unique event identifier (PK)
`event_type`	VARCHAR(20)	—	Change type: `CREATED`, `UPDATED`, or `DELETED`
`entity_type`	VARCHAR(50)	—	Entity that changed: `store`, `product`, `label`, `accesspoint`
`entity_key`	JSONB	—	Composite business key identifying the record (e.g., `{"retail_chain_id": "RC001", "store_id": "S001"}`)
`payload`	JSONB	—	Event payload — full snapshot for CREATED, changed-field diffs for UPDATED, empty `{}` for DELETED
`occurred_at`	TIMESTAMPTZ	`NOW()`	When the change was detected (defaults to transaction time)
`status`	TEXT	`'PENDING'`	Delivery status: `PENDING`, `DELIVERED`, or `FAILED`
`delivered_at`	TIMESTAMPTZ	`NULL`	When the event was published to Solace

Primary key: `id` (UUID, database-generated)

Constraint: `CHECK (status IN ('PENDING', 'DELIVERED', 'FAILED'))`

The `id` column uses UUIDs rather than sequential integers. This enables deterministic event identifiers that can be included in published messages for downstream deduplication — consumers receiving the same event twice (at-least-once delivery) can use the UUID to discard duplicates.

The `status` column tracks delivery lifecycle explicitly. The Event Publisher transitions events from `PENDING` to `DELIVERED` (with `delivered_at` timestamp) after successful publication, or to `FAILED` for permanent errors. This explicit status model enables straightforward observability — a simple `GROUP BY status` reveals the outbox health at a glance.

3.2.2. Payload format

CREATED events carry a full snapshot of the record:

```
{
  "event_type": "CREATED",
  "entity_type": "product",
  "entity_key": {"retail_chain_id": "RC001", "store_id": "S001", "item_id": "P001"},
  "payload": {
    "name": "Leite Mimosa 1L",
    "price": 1.29,
    "status": "active",
    ...
  }
}
```

UPDATED events carry only the changed fields, with old and new values:

```
{
  "event_type": "UPDATED",
  "entity_type": "product",
  "entity_key": {"retail_chain_id": "RC001", "store_id": "S001", "item_id": "P001"},
  "payload": {
    "price": {"old": 1.29, "new": 1.49},
    "status": {"old": "active", "new": "updated"}
  }
}
```

DELETED events carry an empty payload — the entity key identifies the deleted record, and the Event Publisher adds `eventId` and `send_date` at publish time:

```
{
  "event_type": "DELETED",
  "entity_type": "product",
  "entity_key": {"retail_chain_id": "RC001", "store_id": "S001", "item_id": "P001"},
  "payload": {}
}
```

3.2.3. Comparison scope

When determining whether a record has changed, the following columns are excluded from comparison:

- Primary key columns — they identify the record, not its content
- Audit columns — `created_at` and `last_updated_at` are set by the database on every upsert and do not reflect a meaningful data change

The source-provided `modification_date` column *is* included in comparisons, as it reflects when the record was last modified in the Vusion system.

3.3. Indexes

Two partial indexes optimise the outbox's two access patterns:

```
CREATE INDEX idx_event_outbox_pending
    ON esl.event_outbox (occurred_at) WHERE status = 'PENDING';

CREATE INDEX idx_event_outbox_delivered
    ON esl.event_outbox (delivered_at) WHERE status = 'DELIVERED';
```

Index	Purpose
`idx_event_outbox_pending`	Event Publisher queries: `SELECT ... WHERE status = 'PENDING' ORDER BY occurred_at`
`idx_event_outbox_delivered`	Retention cleanup: `DELETE ... WHERE status = 'DELIVERED' AND delivered_at < threshold`

Partial indexes keep each index small — the pending index only covers rows awaiting delivery, and the delivered index only covers published rows. As events transition from `PENDING` to `DELIVERED`, they move from one index to the other.

3.4. Retention

A stored function handles cleanup of delivered events:

```
CREATE OR REPLACE FUNCTION esl.cleanup_delivered_events(retention_days INT DEFAULT 7)
RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE deleted BIGINT;
BEGIN
    DELETE FROM esl.event_outbox
    WHERE status = 'DELIVERED'
      AND delivered_at < NOW() - (retention_days || ' days')::INTERVAL;
    GET DIAGNOSTICS deleted = ROW_COUNT;
    RETURN deleted;
END; $$;
```

The function deletes outbox rows with status `DELIVERED` that were published more than `retention_days` ago (default: 7 days) and returns the number of deleted rows. It is designed to be called externally — by `pg_cron`, the Event Publisher, or an application-level scheduler.

Only delivered events are eligible for cleanup. Events with status `PENDING` or `FAILED` are never deleted, regardless of age, to prevent data loss.

3.5. Migrations

Version	Description
V1.0.0.15	Strip composite `{retail_chain_id}.{store_id}` prefixes from existing data; add `retail_chain_id` to `store_sync_state`
V1.0.0.16	Create `event_outbox` table (UUID primary key, status check constraint)
V1.0.0.17	Create partial indexes on `event_outbox` (`occurred_at` for `PENDING`, `delivered_at` for `DELIVERED`)
V1.0.0.18	Create `cleanup_delivered_events` retention function
V1.0.0.19	Add `deletion_date` to labels; add `deleted_at` to products and labels
V1.0.0.20	Increase entity ID columns (`item_id`, `label_id`, `access_points.id`) to VARCHAR(255)
V1.0.0.21	Reset role-level `statement_timeout` set by V13 (see PgBouncer compatibility below)
V1.0.0.22	Add correlation columns (`run_id`, `retail_chain_id`, `store_id`, `pipeline_name`, `entity_type`) and supporting indexes to `records_with_errors`
V1.0.0.23	Drop the `UNIQUE` qualifier on `idx_ap_mac_address` and `idx_ap_serial_number`; both kept as plain B-Tree indexes for hardware-id lookups
V1.0.0.24	Partition `sync_state` / `store_sync_state` retention triggers by `sync_status` (keep last 20 rows per status per partition key)

These migrations follow the same conventions established in Phase 1: table and indexes in separate files, snake_case naming, all objects in the `esl` schema.

3.5.1. PgBouncer compatibility (V1.0.0.13 + V1.0.0.21)

PgBouncer in transaction pool mode rejects any client whose `StartupMessage` carries unknown runtime parameters with `SQLSTATE 08P01`. Two parameters in particular show up in the wild:

Parameter	Sent by	Resolution
`extra_float_digits = 3`	pgJDBC 42.x (Flyway) at startup	Stripped by PgBouncer's `ignore_startup_parameters`; restored server-side via V13's `ALTER ROLE ... SET extra_float_digits = 3`
`statement_timeout`	pgJDBC 42.x (Flyway) at startup	Stripped by PgBouncer's `ignore_startup_parameters`; not restored server-side — V13 originally set it to `'30s'`, V21 resets it (see below)

V13's `statement_timeout = '30s'` line was a copy-paste of a JDBC-compat recipe and turned out to be a footgun: it pinned every future Flyway migration to a 30s server-side cap, which silently fails any legitimately long DDL (`CREATE INDEX CONCURRENTLY` on a large `esl.labels`, for example). V21 issues `ALTER ROLE %I RESET statement_timeout` against the same `current_user` V13 wrote to. After V13 + V21 apply, `pg_roles.rolconfig` for the migration user is `{extra_float_digits=3}` with no `statement_timeout`.

Per-query timeouts in the Go services are enforced via `context.Context` deadlines in application code, not via a server-side cap — so V21 sets no replacement value. The same reasoning is documented on `commonpg.NewPool`'s godoc.

The companion change in `esl-common` removed `RuntimeParams["search_path"]` from pool creation (it tripped the same 08P01 for every Go service); every consumer now qualifies its SQL via `commonpg.Schema` rather than relying on session `search_path`. See the `search-path-to-schema-qualified` plan for the full PR sequence.

3.5.2. Deletion columns (V1.0.0.19)

Products and labels support soft-delete via dual timestamps:

Column	Table	Description
`deletion_date`	products, labels	Business timestamp from Vusion — when the record was marked deleted. Reliably populated for products; may be NULL for labels (Vusion returns NULL even on confirmed-DELETED label rows)
`deleted_at`	products, labels	Audit timestamp — when the pipeline first detected the deletion. Set by `injectAuditTimestamps` on records with `status = 'DELETED'`; preserved on subsequent upserts via `COALESCE`

Stores and access points have no deletion lifecycle in Vusion and do not have these columns.

3.6. Key Design Decisions

3.6.1. Single outbox table for all entity types

All entity types share one `event_outbox` table rather than having per-entity outbox tables. The `entity_type` column distinguishes between them. This simplifies the Event Publisher (one polling query), keeps migration count low, and avoids schema proliferation as new entity types are added.

3.6.2. JSONB for entity_key and payload

Both `entity_key` and `payload` use JSONB rather than typed columns. This accommodates the varying composite key structures across entity types (2-column key for stores, 3-column key for products) and the variable shape of diff payloads without requiring schema changes.

3.6.3. UUID primary key over sequential integer

A UUID primary key (`gen_random_uuid()`) was chosen over `BIGSERIAL` to provide deterministic event identifiers. The UUID is included in the published Solace message, enabling downstream consumers to deduplicate events under at-least-once delivery semantics — if the Event Publisher crashes after publishing but before marking the row as `DELIVERED`, the same event may be published again on restart, and consumers can detect the duplicate by its UUID.

3.6.4. Explicit status column over NULL-based tracking

An explicit `status` column (`PENDING` → `DELIVERED` / `FAILED`) was chosen over using `delivered_at IS NULL` as the delivery indicator. The explicit model provides clearer semantics, enables a `FAILED` state for permanent errors, and simplifies observability — `SELECT status, COUNT(*) FROM event_outbox GROUP BY status` gives a complete health snapshot.

3.6.5. Partial indexes over full indexes

Full indexes on `occurred_at` or `delivered_at` would include all rows. Partial indexes are more efficient because the outbox has a natural lifecycle: rows start as `PENDING` (matched by the pending index) and transition to `DELIVERED` (matched by the delivered index), then are eventually deleted by the retention function.

3.6.6. Retention via function, not trigger

Unlike Phase 1's `sync_state` tables which use `AFTER INSERT` triggers for retention, the outbox uses an explicitly called function. This avoids adding cleanup overhead to every pipeline transaction — retention runs on its own schedule, independent of the write path. The trigger-based bounds for `sync_state` and `store_sync_state` are documented in Run-history retention.

3.7. Two state tables: `sync_state` vs `store_sync_state`

Two Phase-1 tables track sync state at different grains and both carry a `sync_status` column — a common source of confusion. The sections below describe each in detail.

	`sync_state` (V1.0.0.12)	`store_sync_state` (V1.0.0.14)
Grain	One row per pipeline run	One row per `(pipeline_name, retail_chain_id, store_id)` sync
Status values	`running` → {`success`, `failed`, `cancelled`} (Phase 2)	`success` / `failed` / `cancelled`
Linked by	`id` is the run identity	`run_id` references `sync_state.id` (no FK — see below)
Retention	Last 20 per `(pipeline_name, sync_status)`	Last 20 per `(pipeline_name, retail_chain_id, store_id, sync_status)`

3.8. `store_sync_state` semantics

The Phase 1 `store_sync_state` table is unchanged structurally in Phase 2 but its `sync_status` column is now sourced from sink truth, not connector enqueue:

Status	Trigger
`success`	Streaming finished AND every emitted record was acked with no persistence error
`failed`	Connector-side fetch error, OR sink-side persistence error (constraint violation, batch rollback), OR an "ack leak" where streaming finished but the pending counter is non-zero
`cancelled`	Streaming for the store never finished because another store's failure aborted the run

Practically, this means a store whose labels triggered a unique-constraint violation in the sink now correctly surfaces as `failed` (with the PG error in `error_message`) rather than `success`. The watermark for that store does not advance, so the next run will retry the failed records.

The `*_processed` columns (`products_processed`, `labels_processed`, `access_points_processed`) reflect records that persisted in the destination, not records emitted to the sink channel.

3.9. `records_with_errors` correlation columns

`esl.records_with_errors` gains five nullable columns so failed-record rows can be traced to the run, store, and pipeline that produced them:

Column	Type	Description
`run_id`	BIGINT	Foreign-key-style reference to `sync_state.id` (no FK constraint — see below)
`retail_chain_id`	VARCHAR(255)	Chain the failed record belonged to, post-normalizer
`store_id`	VARCHAR(255)	Store the failed record belonged to. Stripped form (`000010`) — matches `products.store_id` / `labels.store_id`, not the composite `{retail_chain}.{store}` returned by Vusion
`pipeline_name`	VARCHAR(255)	Sourced from the orchestrator's pipeline config (set on the sink at construction)
`entity_type`	VARCHAR(32)	Free-form, populated from `record.Metadata["type"]` (`store`, `product`, `label`, `accesspoint`)

Two indexes back the common query patterns:

```
CREATE INDEX idx_records_with_errors_run_id
    ON esl.records_with_errors (run_id);
CREATE INDEX idx_records_with_errors_store
    ON esl.records_with_errors (run_id, retail_chain_id, store_id);
```

All five columns are nullable. There are two reasons:

1. Backward compatibility. Pre-PR-3 rows pre-date the columns and exist with all five NULL. Dashboards filtering on `run_id` must handle NULL.

2. Tolerance for partial state. A failure that occurs before the sink learns the run's id (e.g. the orchestrator couldn't insert the running row because the state DB was briefly unreachable) still produces a usable `records_with_errors` entry — error recording must never depend on state-store latency.

There is no foreign key from `records_with_errors.run_id` to `sync_state.id`. Error recording must survive a `sync_state` row never being inserted at all (the running-row insert is best-effort), so coupling the two via FK would create a failure mode where a sink-side error vanishes because the state-side write never happened.

`entity_type` is intentionally not constrained to an enum at the database level. The existing `table_name` column already serves as the strict identifier; `entity_type` is the looser, normalizer-level label, kept free-form for forward compatibility with new entity types.

3.10. `sync_state` lifecycle

Phase 2 PR 3 introduces a running → terminal lifecycle on `esl.sync_state`. The status enum now covers four values:

```
running | success | failed | cancelled
```

`sync_state.sync_status` is declared as `VARCHAR(20)` with no database-level CHECK constraint (V1.0.0.12). The enum is enforced by the orchestrator — it is the only writer of this column, and the lifecycle below describes the only paths that move a row between states. Adding a CHECK constraint would be safe (all four current values are ≤ 9 chars) but is intentionally deferred so the column does not need a constraint-rotating migration when a future status is introduced.

Status	When the row is in this state
`running`	The orchestrator inserted the row at the start of a pipeline run; the run hasn't finished yet
`success` / `failed` / `cancelled`	Terminal outcomes — the orchestrator updated the row at the end of the run

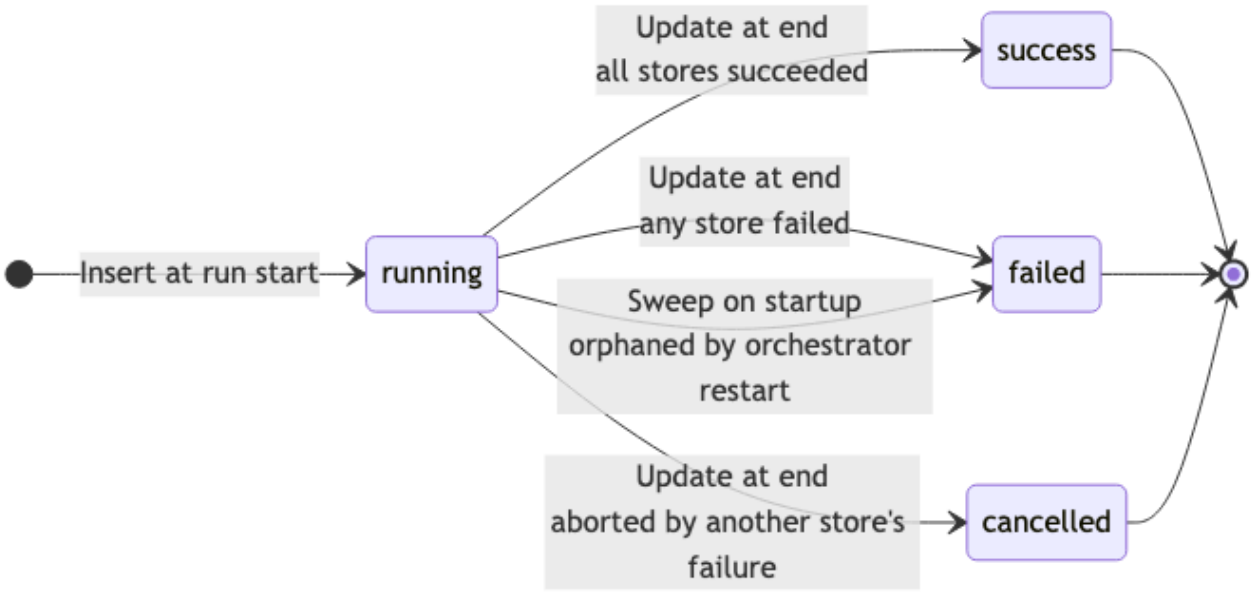


Figure 3: sync_state lifecycle

The orchestrator runs three operations against `sync_state` per run:

1. Sweep on startup. `UPDATE sync_state SET sync_status = 'failed', error_message = 'orphaned by orchestrator restart', finished_at = NOW() WHERE pipeline_name = $1 AND sync_status = 'running'`. Any row left in `running` is presumed dead — its orchestrator must have crashed before reaching the terminal-update step. Pipeline-name uniqueness is the safety boundary; concurrent orchestrators on the same pipeline-name would race here, but that scenario is documented as out of scope.
2. Insert at start. A new row is inserted with `sync_status = 'running'`, zero counts, and `finished_at` set to the run's start time as a placeholder.
3. Update at end. The same row is finalized via `UPDATE sync_state SET sync_status = ..., counts ..., duration = ..., finished_at = ..., error_message = ... WHERE id = $1`. The row's `id` is the `run_id` written into `records_with_errors` for any record that failed during the run.

A `running` row is observable while a pipeline executes — useful for live dashboards but worth being aware of when alerting on `sync_status = 'failed'` (a healthy in-flight run is `running`, not yet `success`). Time-windowed alerts should restrict to `finished_at` within the window, not `started_at`.

3.11. Run-history retention

Both run-history tables are bounded by `AFTER INSERT FOR EACH ROW` triggers — `trg_sync_state_retention` and `trg_store_sync_state_retention`. Each trigger keeps the most recent 20 rows per partition, ordered by terminal timestamp (`finished_at` for `sync_state`, `synced_at` for `store_sync_state`):

Table	Partition key
sync_state	(pipeline_name, sync_status)
store_sync_state	(pipeline_name, retail_chain_id, store_id, sync_status)

The sync_status term was added in V1.0.0.24. Phase 1's partition keys omitted it (one bucket per pipeline, or per store), which was acceptable when the column was effectively binary. Once Phase 2 PR 3 introduced the running → {success, failed, cancelled} lifecycle on sync_state, a streak of success rows in one partition could evict every historical failed or cancelled row before an operator saw it. Partitioning by status preserves the latest 20 of each, multiplying the steady-state row count by the number of distinct statuses observed for the partition — e.g. a pipeline that has only produced success and failed rows caps at 40 sync_state rows; with all four statuses it caps at 80.

The store-side idx_store_sync_state_lookup index (recreated in V1.0.0.15) already covers the new store_sync_state retention query exactly — (pipeline_name, retail_chain_id, store_id, sync_status, synced_at DESC). The sync_state partition lookup is served by idx_sync_state_pipeline (pipeline_name, finished_at DESC), which does not include sync_status; at the table's bounded size (≤ a few thousand rows in steady state) the extra scan is trivial.

4. Data Pipeline — CDC

4.1. Overview

Phase 2 adds Change Data Capture (CDC) to the Data Pipeline's PostgreSQL sink. When enabled, the sink detects CREATED, UPDATED, and DELETED records during each batch write and inserts corresponding change events into the event_outbox table — atomically, within the same database transaction as the upserts.

The CDC logic is encapsulated in a dedicated CDC struct that the sink delegates to. When CDC is disabled (the default), the sink behaves exactly as in Phase 1 — no transaction wrapper, no pre-fetch, no outbox writes.

4.2. Data Normalization

The Vusion API returns composite store IDs in the format {retail_chain_id}.{store_id} (e.g., continente_pt.000010). In Phase 1, these composites were stored as-is in the store_id column — redundant, since retail_chain_id is already a separate column.

In Phase 2, the connector strips the retail chain prefix at ingestion before building records. Only the store portion (000010) is persisted. The composite value is retained internally for VLink API URL paths (/stores/{composite}/productLabelling/products), which require it, but never leaves the connector layer.

Database migration V1.0.0.15 performs a one-time fix-up:

- Strips the `{chain}.` prefix from existing `store_id` values in all entity tables (`stores`, `products`, `labels`, `access_points`) and in `event_outbox.entity_key`
- Adds `retail_chain_id` to `esl.store_sync_state` (previously keyed on `store_id` alone)

The `retail_chain_id` addition to `store_sync_state` prevents cross-chain collisions: with composite IDs stripped, two retail chains sharing the same store code (`000010`) would otherwise overwrite each other's sync timestamps. Sync state is now tracked per `(retail_chain_id, store_id)` pair.

4.3. Feature Flag

CDC is gated behind a single configuration flag:

```yaml
sink:
  postgres:
    cdc:
      enabled: false   # default
```

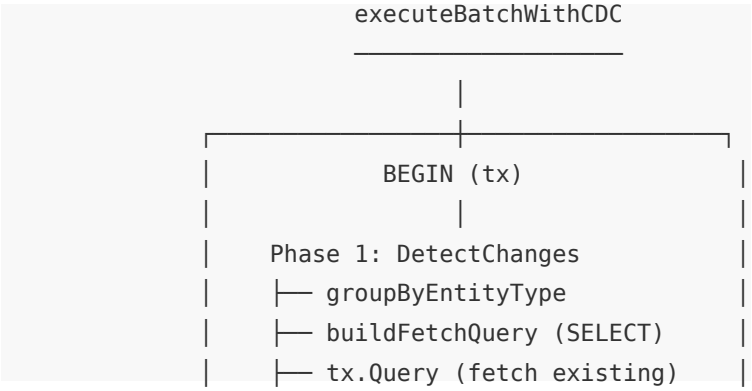
When `cdc.enabled` is `false`, the sink dispatches batches through the original non-transactional path. When `true`, the sink wraps each batch in a transaction and delegates change detection and outbox writes to the CDC module.

The flag also gates the VLink connector's `deleted=true` query param: when CDC is on, list calls include soft-deleted rows so DELETED events can be emitted; when CDC is off, the param is omitted and deleted rows are skipped at the source. This keeps both ends consistent — there is no value in fetching deleted rows the sink would discard.

The flag is evaluated once at startup in the sink builder — the `CDC` struct is only instantiated when enabled. At runtime, the dispatch check is a nil pointer comparison (`s.cdc != nil`), not a config lookup.

4.4. Architecture

All CDC logic lives in a single file (`internal/sink/postgres/cdc.go`) as an encapsulated struct with two public methods. The sink orchestrates the transaction and delegates each phase:



```
│    └── classifyAndDiff                │
│                  │                    │
│    Phase 2: Upsert                    │
│    ├── buildUpsertBatch               │
│    ├── tx.SendBatch                   │
│    └── sendAndProcessBatch            │
│                  │                    │
│    Phase 3: WriteOutbox               │
│    ├── buildOutboxInsert              │
│    └── tx.Exec                        │
│                  │                    │
│            COMMIT                     │
└───────────────────────────────────────┘
```

4.4.1. CDC Struct

```
type CDC struct {
    logger      *zap.Logger
    metrics     *telemetry.Metrics
    schema      commonpg.Schema
    outboxTable string
}
```

The struct carries the schema binding from `esl-common` and pre-computes the schema-qualified outbox identifier (e.g. `"esl"."event_outbox"`) once at construction, so every event INSERT carries an explicit `schema.table` reference rather than relying on `search_path` resolution. It exposes two methods:

- **`DetectChanges`** — fetches existing state within the transaction, classifies each record, and returns a slice of `event.ChangeEvent`
- **`WriteOutbox`** — inserts change events into the outbox within the same transaction

4.4.2. Shared Helpers

Two functions were extracted from the original `executeBatch` to avoid duplication between the CDC and non-CDC paths:

Helper	Purpose
`buildUpsertBatch`	Builds a `pgx.Batch` with upsert queries for all records
`sendAndProcessBatch`	Walks the batch results and returns per-record outcomes aligned with the input records, plus the classified trigger error

Both paths call `sendAndProcessBatch` with the same signature. The non-CDC path additionally runs a per-record fallback after a batch failure (see Error Handling). The CDC path collapses any failure into uniform `rolledBackErr` outcomes after the rollback.

4.5. Change Detection

4.5.1. Step 1 — Group by Entity Type

Records in a batch may belong to different entity types (stores, products, labels, access points). The first step groups them using the `type` field from record metadata:

```
type entityGroup struct {
    tableName    string
    conflictKeys []string
    records      []*models.Record
}
```

Conflict keys come from `entity.ConflictKeys` in esl-common — the single source of truth for entity business keys. Records whose entity type is not present in `entity.ConflictKeys` are silently skipped (no business key definition means CDC cannot build a meaningful event).

4.5.2. Step 2 — Fetch Existing State

For each entity group, CDC builds a SELECT query that fetches the current row values for all records in the group:

```
SELECT "name"::TEXT, "price"::TEXT, "status"::TEXT, ...
FROM esl.products
WHERE ("retail_chain_id", "store_id", "item_id") IN (($1,$2,$3), ($4,$5,$6), ...)
```

Key aspects:

- Explicit column list — derived from the incoming record's `RawData` keys. No `SELECT *` — only columns being upserted are fetched, avoiding false diffs from database-only columns.
- **`::TEXT`** casting — all values are cast to TEXT to avoid Go type mismatches (e.g., `int32` vs `float64`, `time.Time` vs `string`). Both sides are normalised to strings for comparison. NUMERIC columns receive an additional decimal-canonicalization pass (see Value normalisation below) so that scale differences between the stored form and the source representation do not produce spurious diffs.
- Single round-trip — one batched SELECT per entity type, using a composite `IN` clause.

The query runs within the transaction. Under READ COMMITTED isolation (the PostgreSQL default), this is safe because the Data Pipeline is the sole writer to entity tables — no concurrent modifications can occur between the SELECT and the subsequent upsert.

4.5.3. Step 3 — Classify and Diff

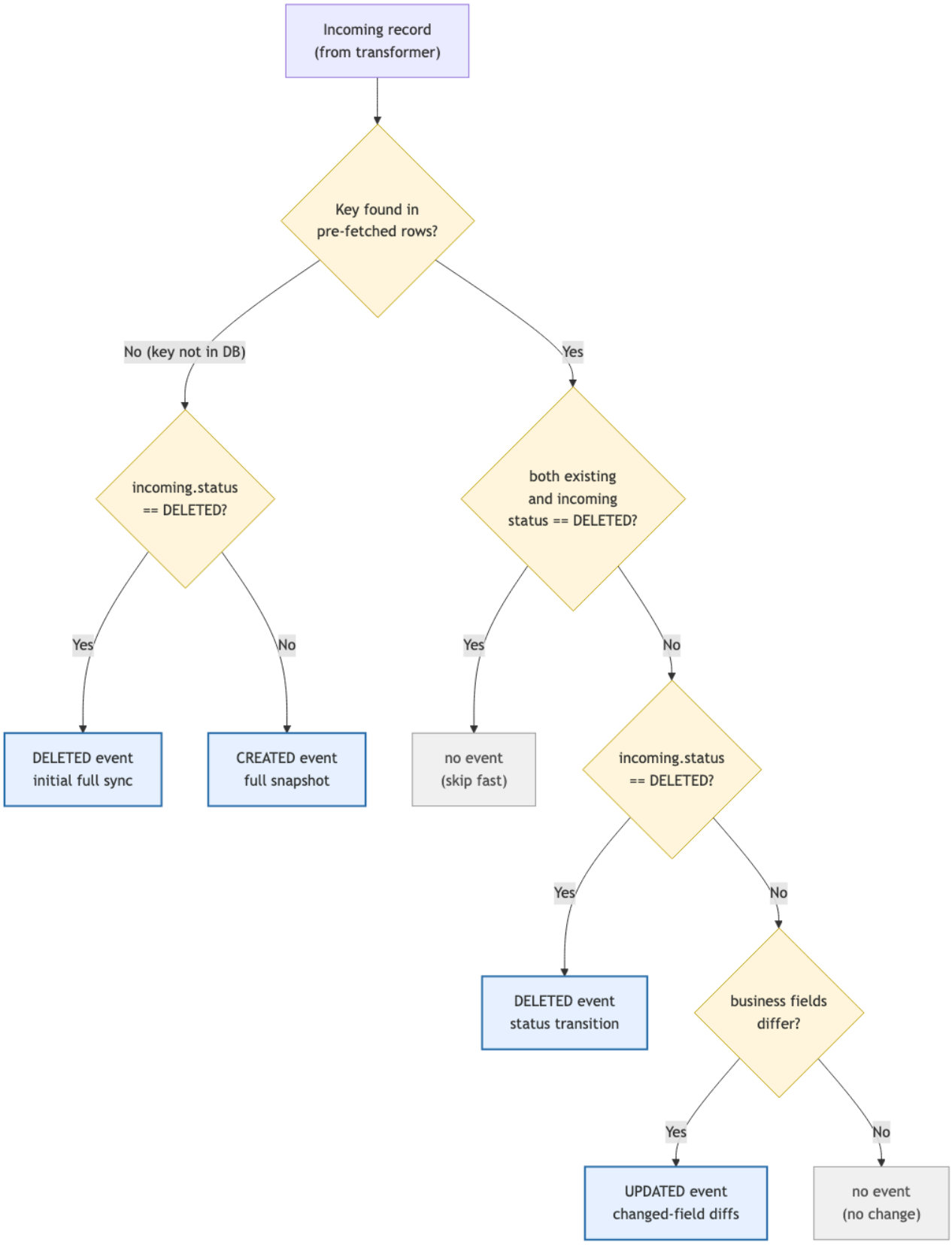


Figure 4: CDC Classifier Flow

Each incoming record is compared against the pre-fetched existing state:

Condition	Event	Payload
Existing and incoming both `status = "DELETED"`	No event (skip fast)	—
Incoming `status = "DELETED"` (transition or first-contact)	DELETED	Empty `{}`
Key not found in existing rows	CREATED	All non-key fields (full snapshot)
Key found, business fields differ	UPDATED	Only changed fields as `{"old": X, "new": Y}` diffs
Key found, all fields identical	No event	—

4.5.3.1. Column handling

Column category	Compared?	CREATED payload	UPDATED payload
Conflict keys (e.g., `store_id`)	No	No (in `entity_key`)	No (in `entity_key`)
Audit columns (`created_at`, `last_updated_at`)	No	Yes (flat value)	Yes (flat value, not diffed)
Business fields	Yes	Yes	Only changed fields

Audit columns are excluded from comparison because they change on every upsert. However, they are included in the event payload: CREATED events carry both timestamps, and UPDATED events include `created_at` (from the existing row) and `last_updated_at` (from the incoming batch) as flat values — not as old/new diffs.

4.5.3.2. Value normalisation

Both sides (incoming `RawData` values and database TEXT values) are normalised to strings before comparison:

Go type	Normalised form
`string`	As-is
`float64`	Formatted as number; trailing `.0` stripped for whole numbers
`bool`	`"true"` / `"false"`
`nil`	`"<nil>"`
`[]any`	Sorted, JSON-marshalled
Timestamps	Parsed and normalised (RFC3339, RFC3339Nano, and Postgres TEXT formats are all equivalent)
Postgres arrays	Parsed, sorted, and emitted as a canonical JSON array string (`["a","b"]`) — same form as the `[]any` row above, so a pristine slice in `RawData` compares equal to a `::TEXT`-cast array literal from the database

This normalisation ensures that semantically equal values are never flagged as changes — for example, a Postgres TEXT timestamp `2024-10-04 08:31:38.879+00` matches an incoming RFC3339 value `2024-10-04T08:31:38.879Z`.

4.5.3.3. NUMERIC columns

NUMERIC columns require an extra step. The `::TEXT` cast preserves the column's declared scale (e.g., `8.00` for `NUMERIC(12,2)`), while the source value may arrive as a JSON number (`8` → Go `float64`) or a JSON string with a different scale (`"8.0"`). Without explicit handling, every record in every batch would emit an UPDATED event purely for scale or type formatting differences.

To avoid this, NUMERIC columns are declared per entity type and routed through a decimal-canonicalization pass that:

- Trims trailing zeros after the decimal point (`8.00` → `8`, `49.90` → `49.9`)
- Removes an empty fractional part (`8.` → `8`)
- Collapses negative zero (`-0` → `0`)
- Accepts equivalent inputs across `float64`, `int`, `int64`, JSON numbers, and decimal-formatted strings

So `8.00` (DB), `8` (JSON number), and `"8.0"` (JSON string) all reduce to the same canonical form and compare equal. The declaration is verified at sink startup against the live `information_schema`: a warning is logged if a declared column is missing from the schema, or if the schema contains a NUMERIC column not declared in the registry.

This handling is applied only to declared columns. String columns whose values happen to look numeric (e.g., zero-padded codes such as `"009648"`) are not affected, so legitimate value differences in non-NUMERIC columns continue to surface as UPDATED events.

4.5.3.4. Payload value normalisation

Values included in event payloads go through a separate normalisation step (`normalizePayloadValue`) that converts strings sourced from `::TEXT`-cast columns back to native Go types before JSON-marshalling:

- Timestamps become `time.Time` and serialise as RFC3339Nano, so an outbox payload sees the same timestamp shape regardless of whether the value came from pristine `RawData` (incoming JSON) or from a database fetch (Postgres TEXT format).
- Postgres array literals become `[]any` and serialise as proper JSON arrays (`["a","b"]`) — required because UPDATE event payloads carry an `old` value sourced from the database (PG literal) alongside a `new` value sourced from pristine `RawData` (real Go slice). Without this normalisation, `old` would round-trip into JSONB as a quoted string while `new` would be a real array.

This keeps the JSONB payload shape uniform across CREATED and UPDATED events.

4.6. Outbox Write

If any changes were detected, CDC builds a single multi-row INSERT:

```
INSERT INTO event_outbox (event_type, entity_type, entity_key, payload, occurred_at)
VALUES ($1,$2,$3,$4,$5), ($6,$7,$8,$9,$10), ...
```

The `entity_key` and `payload` fields are JSON-marshalled before insertion. The `occurred_at` timestamp is the batch timestamp — a deterministic value set once per batch and injected into all records, ensuring the outbox event and the upserted row share the same audit timestamp.

The `id` (UUID), `status` (`PENDING`), and `delivered_at` (`NULL`) columns use their database defaults.

4.7. Transaction Flow

The complete CDC batch execution:

```
BEGIN (READ COMMITTED)
|
├── Phase 1: DetectChanges
|   ├── SELECT existing rows (1 query per entity type)
|   └── Classify: CREATED / UPDATED / unchanged
|
├── Phase 2: Upsert
|   ├── Build pgx.Batch (1 upsert per record)
|   ├── tx.SendBatch → process results (fail-fast)
|   └── On error: ROLLBACK, return failure
|
├── Phase 3: WriteOutbox (only if changes detected)
|   ├── INSERT INTO event_outbox (multi-row)
|   └── On error: ROLLBACK, return failure
```

```
│
└── COMMIT
```

The transaction guarantees atomicity — if any phase fails, all upserts and outbox writes are rolled back. A record is never persisted without its corresponding change event, and a change event is never written without its corresponding data change.

4.7.1. Single-writer assumption

The pipeline uses READ COMMITTED isolation (PostgreSQL default). This is safe because the Data Pipeline is the sole writer to entity tables. No concurrent modifications can occur between the SELECT (Phase 1) and the upsert (Phase 2), so the pre-fetched state is always current.

If another service starts writing to entity tables in the future, this assumption must be revisited — SERIALIZABLE isolation or explicit row-level locking would be needed to prevent missed diffs.

4.8. Performance

4.8.1. Network round-trips

Path	Round-trips	Description
Non-CDC	1	`pool.SendBatch` (auto-commit)
CDC	4–5	BEGIN → SELECT → upsert batch → outbox INSERT → COMMIT

The CDC path adds 3–4 round-trips. On a LAN connection, this translates to approximately 0.1ms per additional round-trip. An earlier design combined SELECT and upserts into a single `SendBatch` call to save one round-trip, but this was dropped — the coupling violated single responsibility and saved only ~0.1ms, not worth the complexity.

4.8.2. Measured overhead

Benchmarks against a real PostgreSQL container (batch sizes 10–500):

Scenario	Overhead vs non-CDC
All unchanged (best case)	~30% — SELECT only, no outbox write
Mixed (10% changed)	~50%
All new (worst case)	~100% — every record produces a CREATED event

At `batch_size=500` (the production default), the worst case adds approximately 4 seconds to a full 240k-record sync. This is acceptable for the current workload profile.

If CDC overhead becomes a bottleneck, the following optimisations are available: - Lower batch size to 200–300 (reduces per-batch transaction scope) - Chunk outbox INSERTs for very large batches - Switch to COPY for bulk outbox loading

4.9. Error Handling

4.9.1. Per-record outcome reporting

The sink reports the persistence outcome of each record back to the pipeline via an `OutcomeHandler` callback installed on the sink at startup:

```
type OutcomeHandler func(record *models.Record, err error)

type Sink interface {
    Write(ctx context.Context, record *models.Record) error
    Close() error
    SetOutcomeHandler(h OutcomeHandler)
    SetRunID(runID int64)
}
```

The contract: sinks must invoke the handler exactly once per record before `Close` returns, with `err == nil` on successful persistence and a non-nil error on permanent failure. The handler is the canonical mechanism for the connector and metrics layer to learn what actually persisted (vs. what was merely emitted to the sink channel).

4.9.2. pgx batch transactional model

`pool.SendBatch` runs all queued queries in an implicit transaction: PostgreSQL aborts the transaction on any error and rolls back even queries that `br.Exec` reported as successful. This means the per-record `Exec` results are not the source of truth on failure — the only safe interpretation of a failed batch is “all records rolled back, none persisted”.

The non-CDC and CDC paths handle this divergence differently because their atomicity contracts differ.

4.9.3. Non-CDC path: per-record fallback

When `executeBatch` runs against the pool and the batch fails, every record is re-executed individually via `pool.Exec`. Each runs in its own implicit transaction and gets its true outcome:

- Records with valid data persist on the second attempt.
- The genuine violator(s) fail with their real PG error, classified by the existing error machinery.
- Outcomes are then dispatched per-record to the `OutcomeHandler`.

The fallback's overhead is bounded: it only triggers after a batch failure, only for the failed batch, and the happy path stays fully pipelined. Records past the trigger no longer carry the cached pgx cascade error in their outcomes — each record's outcome reflects whether it actually persisted.

4.9.4. CDC path: uniform rolledBackErr

`executeBatchWithCDC` wraps the batch, change detection, and outbox writes in a single explicit transaction. Per-record retries outside this transaction would break the upsert/outbox atomicity contract, so the CDC path does not run the fallback. Instead:

- Commit succeeds → every record's outcome is `nil`.
- Anything fails (`DetectChanges`, batch upsert, `WriteOutbox`, or commit itself) → every record's outcome carries `rolledBackErr`, which wraps the underlying root cause via `%w`. Downstream callers can use `errors.As` to recover the original `*pgconn.PgError` for classification.

The `rolledBackErr` synthesises a uniform per-record error from a single source of truth (the commit result), avoiding the need to interpret poisoned `br.Exec` results.

4.9.5. Outcome-driven store completion

The pipeline tracks a per-store pending counter to detect ack leaks and to drive store-level status classification. The flow:

- `RecordEmitted(storeID)` is called after every record successfully enters the sink-bound channel and increments the store's pending counter.
- `RecordPersisted(record, err)` is the body of the OutcomeHandler. It decrements pending, increments either the per-store persisted counter (on success) or the per-store persistence-failure counter (on failure), and notifies the Phase 1 store gate when the record is store-typed.
- `MarkStoreStreamingDone(storeID)` is called when Phase 2 finishes streaming records for a store (`processStoreEntities` returns nil).

A store reaches the success branch only when streaming finished AND every emitted record was acked. A non-zero pending counter after `sink.Close` always signals a code defect — either a pipeline-side drop site that forgot to report or a sink that violated the OutcomeHandler contract — and surfaces as `failed: ack leak: N records unaccounted for` in the per-store run.

The Phase 1 store gate is now metrics-driven. The connector emits N store records, then calls `WaitForStoreRecords(ctx, N)`, which blocks until N store-typed records have been reported via `RecordPersisted`. `Record.Ack` is gone; one shared OutcomeHandler installed by the pipeline builder is the only mechanism.

4.9.6. Per-store status resolution

`failureStatusResolver` walks four signals in priority order:

1. Connector-side error (e.g. failed Vusion API fetch) → `failed` with the connector error message.

2. Sink-side persistence error (first message wins per store) → `failed` with `"persistence: <msg>"`.
3. Streaming never finished → `cancelled: another store failed`.
4. Streaming finished but pending != 0 → `failed: ack leak: N records unaccounted for`.

Otherwise the store is `success`. The same resolver is used on both the success and failure pipeline-run paths so a store with persistence failures cannot masquerade as `success` even when the global pipeline finished cleanly.

4.9.7. `*_processed` columns: persisted, not emitted

`StoreSyncRun.{Products,Labels,AccessPoints}Processed` now reflects records that actually persisted in the destination, not records emitted to the sink channel. The previous "emitted" semantics conflated successful enqueue with successful write — a store with constraint-violating labels would report `labels_processed = 427` while only ~80 rows landed in the labels table. The honest count comes from `GetStorePersistedMetrics`, populated by the OutcomeHandler.

4.9.8. Records-with-errors accuracy

Only records that genuinely fail contribute to the `records_with_errors` table. Specifically:

- Non-CDC: only the records whose individual fallback `pool.Exec` fails are inserted. The cascade no longer produces false positives for records past the trigger.
- CDC: only the trigger record (or the genuine source of failure, e.g. an `DetectChanges` query error) is inserted, since the rollback prevents any record from being persisted.

4.9.9. Records-with-errors correlation

Each row inserted into `records_with_errors` carries five correlation columns that pin the failure to a specific run, store, and pipeline:

Column	Source
`run_id`	Sink's runID field, populated via `SetRunID(int64)` at run start (after `InsertRun` returns)
`retail_chain_id`	`record.RawData["retail_chain_id"]` — the post-normalizer field name
`store_id`	`record.RawData["store_id"]` — the stripped form, matching `products.store_id`. The `{retail_chain}.{store}` composite returned by Vusion is stripped at the connector boundary and never reaches the sink
`pipeline_name`	Sink's pipelineName field, set once via `WithPipelineName(name)` on the sink builder
`entity_type`	`record.Metadata["type"]` (`store`, `product`, `label`, `accesspoint`)

SetRunID is a setter, not a constructor parameter or context value. A setter keeps the sink reusable across runs (no per-run rebuild), keeps the dependency explicit at the call site (no `ctx.Value` lookups), and is trivial to mock in tests. The trade is that callers must remember to invoke it; missing the call is non-fatal — `run_id` ends up NULL via `NULLIF($n, 0)`, and downstream queries already need to handle NULL for backward compatibility with pre-PR-3 rows.

`pipeline_name` follows a different shape because it doesn't change between runs: it's set on the sink builder via `WithPipelineName(name)` and stays put for the sink's lifetime. Same NULL-tolerant treatment applies if the builder option is omitted.

Typical correlation queries:

```
-- All failed records for one run, grouped by store
SELECT retail_chain_id, store_id, entity_type, COUNT(*)
  FROM esl.records_with_errors
 WHERE run_id = $1
 GROUP BY retail_chain_id, store_id, entity_type;

-- Recent failed records for a specific store (across runs)
SELECT run_id, created_at, error_message
  FROM esl.records_with_errors
 WHERE retail_chain_id = $1 AND store_id = $2
 ORDER BY id DESC LIMIT 50;
```

4.9.10. Error classification

The same error classification from Phase 1 applies:

Type	Action
Transient (deadlock, serialisation, system errors)	Retried with exponential backoff
Permanent (constraint violations, syntax errors)	Stored in `records_with_errors`, not retried

4.10. Observability

4.10.1. Metrics

Metric	Type	Description
`sink.cdc.events` (attribute: `event_type=CREATED`)	Counter	Number of CREATED events detected
`sink.cdc.events` (attribute: `event_type=UPDATED`)	Counter	Number of UPDATED events detected
`sink.cdc.events` (attribute: `event_type=DELETED`)	Counter	Number of DELETED events detected
`sink.cdc.detect_duration_ms`	Histogram	Time spent in change detection (SELECT + classify)
`sink.cdc.outbox_duration_ms`	Histogram	Time spent writing to the outbox

All metrics include the `sink.type=postgres` attribute and are prefixed with `eslorchestrator.` in the OTel exporter.

4.10.2. Logging

Change detection (INFO level):

```
{
  "msg": "CDC change detection completed",
  "batch_size": 100,
  "cdc_created": 3,
  "cdc_updated": 7,
  "cdc_unchanged": 90,
  "cdc_fetch_ms": 12
}
```

Outbox write (DEBUG level):

```
{
  "msg": "CDC outbox write",
  "event_count": 10,
  "cdc_outbox_ms": 2
}
```

4.11. Configuration Reference

CDC adds a single configuration block under `sink.postgres`:

```
sink:
  postgres:
```

```
  # ... existing connection and batching settings ...

  cdc:
    enabled: true   # Enable CDC change detection and outbox writes
```

No additional CDC-specific settings are needed. The outbox table must exist in the database (see Database migrations V1.0.0.16–18).

4.12. Deletion Detection

4.12.1. Scope

DELETED event detection applies to products and labels only. Stores and access points have no deletion lifecycle in Vusion.

4.12.2. VLink `deleted=true`

The VLink client always passes `deleted=true` on all product and label endpoints. This is an additive toggle — the response includes both ACTIVE and DELETED rows in a single paginated walk. No separate endpoint or second request is needed.

4.12.3. Detection signal

The canonical detection signal is `status == "DELETED"`. This works for both products and labels. `deletionDate` from Vusion is stored as informational metadata but is not used for detection — it is unreliable for labels (observed NULL on confirmed-DELETED rows).

4.12.4. Soft-delete storage

Records are soft-deleted via dual timestamps following the existing audit convention:

Column	Source	Description
`deletion_date`	Vusion `deletionDate` field	Business timestamp — when Vusion marked the record deleted. Reliable for products; informational for labels (may be NULL)
`deleted_at`	Pipeline batch timestamp	Audit timestamp — when the pipeline first detected the deletion. Set by `injectAuditTimestamps` regardless of CDC on/off. Preserved on subsequent upserts via `COALESCE(existing, new)`

The record stays in the database with all fields intact. No hard `DELETE` SQL is issued.

4.12.5. Audit timestamp (`deleted_at`) behaviour

`deleted_at` is injected by `injectAuditTimestamps` — the same function that sets `created_at` and `last_updated_at` on every record. When a record has `status == "DELETED"`, `deleted_at` is set to the batch timestamp. This works identically with CDC enabled or disabled.

On upsert conflict, the SQL uses `COALESCE(existing.deleted_at, EXCLUDED.deleted_at)` — the first-detection timestamp is preserved; subsequent syncs of the same deleted record do not overwrite it.

4.12.6. Classification edge cases

Scenario	Behaviour
ACTIVE → DELETED	Emit DELETED event (empty payload)
DELETED → DELETED	No event (skip fast)
DELETED → ACTIVE (theoretical undelete)	Treated as regular UPDATED — no special event type
Key not in DB + incoming DELETED	Emit single DELETED event (initial full sync)

4.13. Known Limitations

- Labels **`deletion_date`** may be NULL on confirmed-DELETED rows. Vusion populates the `deletionDate` field unreliably for labels (observed NULL on rows whose `status` is `DELETED`). The pipeline stores `deletion_date` when provided and leaves it NULL otherwise; it never uses `deletion_date` as a detection signal. `status == "DELETED"` is the canonical classifier, and `deleted_at` (audit timestamp) is always set when the deletion is detected. Consumers that need a reliable deletion timestamp should read `deleted_at`.

5. Event Publisher

5.1. Overview

The Event Publisher is a long-running Go service that polls the `event_outbox` table for pending CDC events and publishes them to a Solace message broker. It runs as a Kubernetes Deployment, decoupled from the Data Pipeline CronJob.

The service follows a simple loop: poll → transform → publish → mark delivered. Each cycle runs within a single database transaction, using `SELECT FOR UPDATE SKIP LOCKED` for row-level locking. Events are published with persistent confirmed delivery, providing at-least-once guarantees.

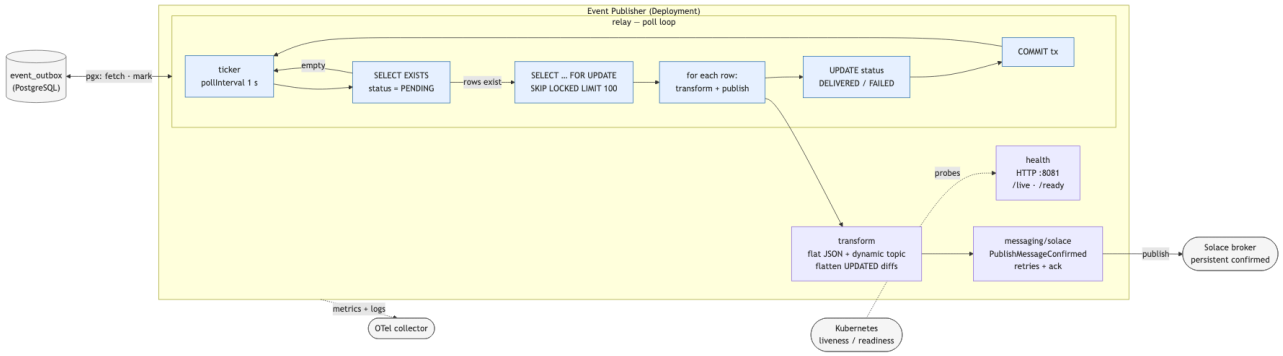


Figure 5: Event Publisher Flow

5.2. Technology Stack

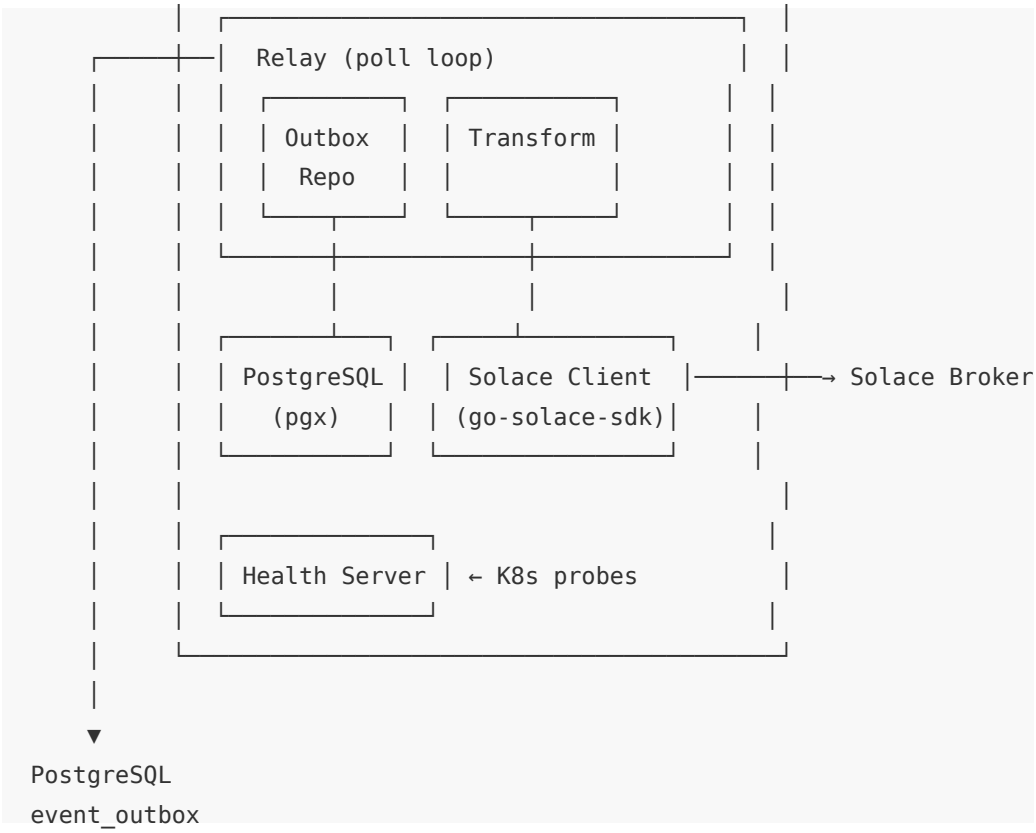
Technology	Purpose
Go 1.26	Application language
pgx/v5 via esl-common	PostgreSQL driver with connection pooling
go-solace-sdk	Solace client library (wraps `solace.dev/go/messaging`)
Viper	YAML configuration with environment variable overrides
Zap	Structured JSON logging
OpenTelemetry	Metrics (application + SDK-level)

5.3. Architecture

The service is organised into five internal packages:

Package	Responsibility
`relay`	Core polling loop, batch processing, transaction management
`outbox`	PostgreSQL repository — fetch pending, mark delivered/failed
`transform`	Outbox row → flat JSON payload + dynamic Solace topic
`messaging/solace`	Solace client wrapper — connect, publish confirmed, disconnect
`health`	HTTP liveness and readiness probe server

```
┌──────────────────────────────────────────┐
│              Event Publisher             │
│                                          │
```



5.4. Polling Loop

The relay runs a ticker-based loop at a configurable interval (default: 1 second). Each cycle:

5.4.1. 1. Check for pending events

A cheap `SELECT EXISTS` query determines whether any `PENDING` rows exist. If the outbox is empty, the cycle ends without opening a transaction.

5.4.2. 2. Fetch and lock rows

```sql
SELECT id, event_type, entity_type, entity_key, payload, status, occurred_at, delivered_at
FROM event_outbox
WHERE status = 'PENDING'
ORDER BY occurred_at
LIMIT $1
FOR UPDATE SKIP LOCKED
```

`FOR UPDATE SKIP LOCKED` acquires row-level locks without blocking. Rows already locked by another transaction are skipped, which naturally supports horizontal scaling — multiple publisher replicas can process distinct rows concurrently without coordination.

5.4.3. 3. Process each row

For each fetched row:

1. Transform — Convert the outbox row into a flat JSON payload and a dynamic Solace topic (see Event Transformation)
2. Publish — Send to Solace with persistent confirmed delivery
3. Track outcome — Collect the row ID into a delivered or failed list

Processing continues even if individual rows fail. Transform errors (unsupported entity type, missing key fields) and publish errors (Solace unreachable, confirmation timeout) both result in the row being marked as `FAILED`, but do not affect other rows in the batch.

5.4.4. 4. Mark outcomes

Two batch updates within the same transaction:

```
UPDATE event_outbox SET status = 'DELIVERED', delivered_at = $1 WHERE id = ANY($2)
UPDATE event_outbox SET status = 'FAILED' WHERE id = ANY($3)
```

5.4.5. 5. Commit

The entire cycle — fetch, status updates — commits atomically. If the transaction fails, all status changes are rolled back and rows return to `PENDING` for the next cycle.

5.5. Event Transformation

The transform step converts an outbox `ChangeEvent` into a flat JSON payload and a Solace topic. The Event Publisher does not send raw outbox rows — it restructures the data for downstream consumers.

5.5.1. Payload assembly

1. Start with entity key fields — `retail_chain_id`, `store_id`, and any additional conflict keys (`item_id`, `label_id`, `id`)
2. Merge payload fields — For CREATED events, the payload is already flat. For UPDATED events, diff objects `{"old": X, "new": Y}` are flattened to extract only the `"new"` value. For DELETED events, the payload is empty — only entity key fields and metadata are published. Audit columns (`created_at`, `last_updated_at`) pass through as-is (they are already flat values, not diffs).
3. Add metadata — `eventId` (the outbox row UUID) and `send_date` (current time in RFC3339)

The result is a flat JSON object regardless of event type. Downstream consumers distinguish between event types via the Solace topic, not the payload structure.

5.5.2. Example published events

CREATED / UPDATED — payload carries entity key fields, business fields, and audit timestamps:

```
{
  "eventId": "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11",
  "retail_chain_id": "continente_pt",
  "store_id": "000010",
```

```
  "store_name": "Loja Alpha",
  "creation_date": "2026-03-01T10:00:00Z",
  "modification_date": "2026-04-06T14:30:00Z",
  "created_at": "2026-03-01T10:00:00Z",
  "last_updated_at": "2026-04-06T14:30:00Z",
  "send_date": "2026-04-09T12:00:00Z"
}
```

DELETED — minimal payload: only entity key fields plus `eventId` and `send_date`. No business fields, no audit timestamps. Consumers identify the deleted record via the entity key and the `deleted` topic segment:

```
{
  "eventId": "b1ffec00-1d2a-4fa8-9c7e-7cc0ce4917bb",
  "retail_chain_id": "bomdia_pt",
  "store_id": "009648",
  "item_id": "5601234567890",
  "send_date": "2026-04-16T09:15:00Z"
}
```

5.6. Solace Publishing

5.6.1. Topic structure

Topics follow a hierarchical pattern:

`in-store/orchestratoresl/{entitySegment}/{messageType}/v1/{insignia}/{storeId}`

Variable	Source
`{entitySegment}`	Static mapping from entity type (see table below)
`{messageType}`	Event type lowercased: `created`, `updated`, `deleted`
`{insignia}`	`retail_chain_id` from `entity_key`
`{storeId}`	`store_id` from `entity_key`

Entity type to topic segment mapping:

Entity type	Topic segment
`store`	`stores/store`
`product`	`products/product`
`label`	`labels/label`
`accesspoint`	`accesspoints/accesspoint`

Example topics:

```
in-store/orchestratoresl/stores/store/created/v1/continente_pt/000010
in-store/orchestratoresl/products/product/updated/v1/continente_pt/000010
in-store/orchestratoresl/products/product/deleted/v1/bomdia_pt/009648
in-store/orchestratoresl/labels/label/created/v1/continente_pt/000010
in-store/orchestratoresl/accesspoints/accesspoint/updated/v1/continente_pt/000010
```

5.6.2. Delivery mode

Events are published using persistent confirmed delivery (`PublishMessageConfirmed`). The Solace broker acknowledges receipt before the publisher considers the message delivered. This prevents data loss during broker-to-consumer distribution.

The go-solace-sdk handles publish retries internally (3 attempts, exponential backoff). If all attempts fail or the confirmation timeout is exceeded, the Event Publisher marks the outbox row as `FAILED`.

5.7. Health Endpoints

The service exposes HTTP endpoints for Kubernetes probes:

Endpoint	Probe	Behaviour
`/health`	Liveness	Always returns `200 OK` if the process is running
`/ready`	Readiness	Returns `200 OK` if both PostgreSQL and Solace are connected; `503 Service Unavailable` otherwise

The readiness check pings the database pool and verifies the Solace client's connection status. If either dependency is down, the service reports not ready and Kubernetes stops routing traffic to it.

Port and paths are configurable via `health.port`, `health.health_path`, and `health.ready_path`.

5.8. Graceful Shutdown

On `SIGINT` or `SIGTERM`:

1. Stop accepting new poll cycles
2. Finish the current batch (publish remaining events, update statuses, commit)
3. Shut down the health HTTP server
4. Disconnect the Solace client
5. Close the database pool

A configurable shutdown timeout (`publisher.shutdown_timeout`, default: 30 seconds) bounds the entire sequence. If exceeded, the process force-exits.

5.9. Configuration Reference

The service is configured via a YAML file with environment variable overrides. Environment variables use uppercase underscore notation (e.g., `DATABASE_HOST`, `SOLACE_TOPIC_PREFIX`).

5.9.1. Database

```yaml
database:
  host: localhost
  port: 5432
  user: esl
  password: ""
  database: eslorchestrator
  schema: esl                          # Schema under which event_outbox lives; every query
qualifies it explicitly
  ssl_mode: disable
  application_name: "event-publisher"
  max_conns: 4
  min_conns: 1
  max_conn_lifetime: "1h"
  max_conn_idle_time: "30m"
  health_check_period: "1m"
  connect_timeout: "5s"
```

All fields map directly to esl-common's `postgres.PoolConfig`.

5.9.2. Solace

```yaml
solace:
  host: tcp://localhost:55555          # tcp:// or tcps:// for TLS
  vpn: default                         # Message VPN name
  auth_scheme: basic                   # "basic" (default) or "oauth2"
  # Basic auth (used when auth_scheme is "basic")
  username: admin
  password: admin
  # OAuth2 client credentials (used when auth_scheme is "oauth2")
  # client_id: ""                      # OAuth2 client ID
  # client_secret: ""                  # OAuth2 client secret
  # token_endpoint: ""                 # OAuth2 token endpoint URL (https://...)
  # scope: ""                          # OAuth2 scope requested when acquiring tokens
  topic_prefix: "in-store/orchestratoresl"
  connect_timeout: "10s"               # TCP connection timeout (min: 1s)
  connect_retries: 3                   # TCP connect retry attempts
  reconnect_retries: 3                # Post-disconnect reconnection attempts (-1 = infinite)
  reconnect_wait: "2s"                 # Delay between reconnect attempts (min: 100ms)
```

```
  keep_alive: "30s"                  # TCP keep-alive interval
  confirmation_timeout: "10s"        # Publish confirmation timeout
```

`auth_scheme` selects how the publisher authenticates with the broker. Local development uses `basic` against the docker-compose Solace container. Deployed environments use `oauth2` (client-credentials flow): the SDK acquires a token from `token_endpoint` and refreshes it automatically. When `auth_scheme: oauth2`, `client_id`, `client_secret`, `token_endpoint`, and `scope` are all required; when `basic` (or unset), `username` is required.

5.9.3. Publisher

```
publisher:
  poll_interval: "1s"                # Outbox polling frequency
  batch_size: 100                    # Max events per poll cycle
  shutdown_timeout: "30s"            # Max time to finish batch on shutdown
```

5.9.4. Health

```
health:
  port: 8081
  health_path: "/health"
  ready_path: "/ready"
```

5.9.5. Logging

```
log:
  level: info                        # debug, info, warn, error
```

5.10. Observability

5.10.1. Application metrics

Metric	Type	Attributes	Description
`event_publisher.events_processed_total`	Counter	`entity_type`, `event_type`, `status`	Events processed, segmented by outcome
`event_publisher.poll_batch_size`	Histogram	—	Events fetched per poll cycle
`event_publisher.poll_cycles_total`	Counter	—	Total poll cycles (including empty)

The `status` attribute on `events_processed_total` distinguishes between `delivered` and `failed` outcomes, enabling alerting on failure rates per entity type.

5.10.2. SDK metrics

The go-solace-sdk automatically records its own metrics when a `MeterProvider` is supplied:

- `solace.core.*` — Connection pool and transport metrics
- `solace.producer.*` — Publish confirmation counts, latency, retries, and payload size

5.10.3. Logging

All log entries use Zap structured JSON. Key log events:

Level	Event	Fields
INFO	Poll cycle completed	`batch_size`, `delivered`, `failed`
INFO	Startup / shutdown	Component states
WARN	Publish failure	`event_id`, `entity_type`, `error`
WARN	Transform failure	`event_id`, `entity_type`, `error`
ERROR	Transaction failure	`error`
DEBUG	Individual event processed	`event_id`, `topic`, `status`

The go-solace-sdk logs are bridged through Zap via a `zapslog.NewHandler` adapter, consolidating all output into a single structured format.

5.11. Delivery Guarantees

The Event Publisher provides at-least-once delivery:

1. Events are published to Solace before the database status is updated
2. If the process crashes between publishing and committing, the outbox row stays `PENDING`
3. On restart, the row is polled again and re-published

Consumers must deduplicate using the `eventId` field in each published event. This is the outbox row's UUID primary key — deterministic across retries. The duplication window is bounded by the batch size (default: 100 events) and only occurs during crash recovery, not during normal operation.

5.12. Key Design Decisions

5.12.1. Polling over WAL logical replication

The Event Publisher polls the outbox table rather than consuming PostgreSQL's WAL (Write-Ahead Log) via logical replication. Both approaches were evaluated:

Concern	Polling	WAL streaming
Operational complexity	Low — standard SQL queries	High — requires `wal_level=logical`, replication slots, WAL retention management
Latency	Configurable (default 1s)	Near real-time (sub-second)
Observability	Free via `status` column — `GROUP BY status` shows outbox health	Requires external monitoring of replication lag and slot state
WAL bloat risk	None	Inactive slot holds WAL indefinitely, risking disk exhaustion
Failure handling	Explicit `FAILED` status in database	Complex — failed messages need a dead-letter mechanism
Scaling	`FOR UPDATE SKIP LOCKED` supports multiple replicas	Requires partitioned replication or a single consumer

Polling was chosen because the 1-second latency is sufficient for the ESL use case, operational simplicity is a priority (no DBA involvement for replication slots), and the explicit `status` column provides built-in observability. WAL streaming would be the better choice for high-throughput or sub-second latency requirements.

5.12.2. Transaction-scoped publishing

The entire poll cycle — fetch, publish, mark — runs within a single database transaction. Rows stay locked during Solace publishing. This is acceptable because:

- The Data Pipeline only INSERTs into the outbox (no conflict with row locks)
- Batch size is capped (100 events)
- A single publisher instance is expected in the initial deployment

5.12.3. No application-level retry for failed events

Events that fail to transform or publish are marked `FAILED` immediately — there is no automatic retry. The go-solace-sdk handles low-level publish retries internally (3 attempts, exponential backoff), but if the SDK exhausts its retries, the event is considered failed.

This design keeps the publisher stateless and simple. Failed events remain in the outbox for investigation. Replay or retry is handled externally (e.g., resetting status to `PENDING` after fixing the root cause).

5.13. Known Limitations

5.13.1. No per-entity ordering guarantee

With a single publisher replica, events for the same entity are published in `occurred_at` order. With multiple replicas, `FOR UPDATE SKIP LOCKED` distributes rows across replicas without entity-key affinity — two events for the same entity could be processed by different replicas and arrive out of order.

If downstream consumers require per-entity ordering, partitioning by entity key (e.g., consistent hashing) would be needed.

5.13.2. Failed events require manual intervention

Events marked `FAILED` stay in the outbox indefinitely. There is no dead-letter queue or automatic retry mechanism. Recovery requires identifying the root cause and resetting the status to `PENDING` manually or via an operational script.

6. Deployment & Infrastructure

6.1. Overview

Phase 2 adds one new component (event-publisher) and updates one existing component (dat-apipeline). All are deployed via the existing single Argo CD Application managed by the ESL Orchestrator Helm chart — no new cluster-level resources or separate pipelines.

The rollout has two goals: (1) chart changes can land safely with CDC disabled, and (2) entity tables are populated by at least one full sync *before* CDC is enabled, so that the first CDC run does not flood Solace with a CREATED event per active row plus a DELETED event per historical DELETED row VLink returns. The CDC flag is flipped per environment only after both conditions are met.

6.2. Component Inventory

Component	Workload	Lifecycle	Phase 1	Phase 2
dbMigrations	Flyway Job (PreSync hook)	One-shot per sync	Existing	New migrations V1.0.0.14–24 (V1.0.0.15 strip composite `store_id` + add `retail_chain_id`; V1.0.0.16–18 `event_outbox` table, indexes, retention function; V1.0.0.19 deletion columns; V1.0.0.20 entity ID length widening; V1.0.0.21 reset role-level `statement_timeout`; V1.0.0.22 `records_with_errors` correlation columns; V1.0.0.23 drop `UNIQUE` qualifier on access-points indexes; V1.0.0.24 partition sync-state retention triggers by `sync_status`)
datapipeline	CronJob	Scheduled (daily default)	Existing	Added CDC logic + `cdc.enabled` flag
datafetch	Deployment + Service + Ingress	Always-on	Existing	Unchanged
event-publisher	Deployment	Always-on	—	New

6.3. Prerequisites

Before deploying Phase 2, the following must be in place:

6.3.1. 1. Vault entries

- `{vaultBasePath}/database` — existing (Phase 1), no changes
- New: `{vaultBasePath}/solace` with two properties (OAuth2 client credentials):
  - `client-id` — OAuth2 client ID issued by the IdP
  - `client-secret` — OAuth2 client secret

The remaining Solace config (`host`, `vpn`, `token_endpoint`, `scope`, `topic_prefix`) is non-sensitive and lives in the per-env values file (`eventPublisher.config.solace.*`). `auth_scheme` is hardcoded to `oauth2` in the ConfigMap — basic auth is only used in local docker-compose and is not wired into the chart.

The event-publisher's ExternalSecret references both `{vaultBasePath}/database` (reuses the same credentials as datapipeline) and `{vaultBasePath}/solace`. Missing entries cause the ExternalSecret reconcile to fail and block pod startup.

6.3.2. 2. Solace Cloud gateway in cluster

The event-publisher's network policy targets `namespace: solace-cloud`, `app: solace`, port `55443`. Confirm this gateway pod exists in the target cluster before applying the chart — its absence silently blocks egress (the publisher will start but fail readiness).

6.3.3. 3. Database migration compatibility

V1.0.0.15 performs a one-time, idempotent fix-up: strips composite `{retail_chain_id}.{store_id}` prefixes from all `store_id` columns and adds `retail_chain_id` to `store_sync_state`. Safe to re-run. See 03-database.md for migration details.

6.3.4. 4. Image built and pushed

The event-publisher image must be built and pushed to `ghcr.io/sonaemc-instore/lac1041-instoreorchestrator_esl-eventpublisher-application` before setting `eventPublisher.imageTag` in the per-env values file.

6.4. Rollout Sequence

Zero-downtime rollout in three gated steps.

6.4.1. Step 1 — ArgoCD sync with CDC disabled

1. Merge the chart update with `dataPipeline.config.cdc.enabled: false` (the default) in the per-env values file
2. ArgoCD picks up the change and executes the PreSync phase first: Flyway runs the new migrations, including V1.0.0.15
3. ArgoCD then applies the regular resources: datapipeline CronJob ConfigMap is updated, event-publisher Deployment is created
4. The event-publisher starts, polls the outbox (empty) and idles at the configured `poll_interval`
5. The next scheduled datapipeline run uses the updated image; with `cdc.enabled: false`, the sink behaves as in Phase 1 — no transaction wrapper, no outbox writes

Outcome: all infrastructure is in place, no change events are produced, no risk of incorrect publishing during migration.

6.4.2. Step 2 — Verify readiness

```
# Event publisher pod is running and ready
kubectl get pods -l app=orchestrator-esl-eventpublisher

# Readiness probe passes (reports DB + Solace connectivity)
kubectl exec -it deploy/orchestrator-esl-eventpublisher -- curl -sf http://localhost:8081/
ready

# Outbox table exists and is empty
psql "$DB_URL" -c "SELECT COUNT(*), status FROM esl.event_outbox GROUP BY status"

# Latest datapipeline run completed cleanly with CDC disabled (no cdc_* log fields)
kubectl logs -l app=orchestrator-esl-datapipeline --tail=500 | grep -i cdc || echo "CDC not
active — expected"
```

6.4.3. Step 3 — Enable CDC

1. Edit `values-{env}.yaml`, set `dataPipeline.config.cdc.enabled: true`
2. Commit, push, and let ArgoCD sync the ConfigMap change
3. Next datapipeline run produces change events into the outbox
4. Event-publisher drains the outbox on each poll cycle

6.4.4. Step 4 — Verify event flow

```
# Outbox shows DELIVERED rows
psql "$DB_URL" -c "SELECT status, COUNT(*) FROM esl.event_outbox GROUP BY status"

# Event publisher metrics show non-zero delivered events
kubectl exec -it deploy/orchestrator-esl-eventpublisher -- \
  curl -s http://localhost:8081/metrics | grep event_publisher_events_processed
```

Spot-check Solace topic subscription via the Solace Cloud portal or a test consumer. Expect traffic on `.../created/v1/...`, `.../updated/v1/...`, and `.../deleted/v1/...` topic segments — a first-contact sync will surface historical DELETED rows on the `deleted` topics (see 04-datapipeline.md and 07-operations.md).

6.5. Rollback

6.5.1. Pause event publishing (keep outbox intact)

1. Set `dataPipeline.config.cdc.enabled: false` in the env's values file and push
2. ArgoCD syncs — next datapipeline run produces no new events
3. event-publisher drains any remaining PENDING rows, then idles
4. No database rollback needed — V1.0.0.15 is data-only and the `event_outbox` table is always safe to leave in place

6.5.2. Fully remove event-publisher

1. Remove (or comment out) the eventPublisher section in per-env values
2. `helm upgrade` or ArgoCD sync — the SA, ExternalSecret, ConfigMap, and Deployment are pruned
3. The outbox table stays — re-enabling the publisher later picks up from where it left off

6.6. Per-Environment Promotion

Env	Order	Gate to flip CDC
dev	1st	Developer-driven: flip CDC on, run sync manually, inspect outbox and published events
pp	2nd	QA validation: events produced match expected shape and counts; outbox drains; Solace topic receives messages
prd	3rd	Ops validation: publisher stable for ≥24h with CDC off; verified telemetry and log flow; stakeholder sign-off before flipping

6.7. Diagrams

6.7.1. Kubernetes topology

Namespace-level view of the Phase 2 workloads and their connections to PostgreSQL, Vault (via ExternalSecret), Solace Cloud, and the OTel collector. The event-publisher (blue border) is the only new workload; everything else existed in Phase 1.

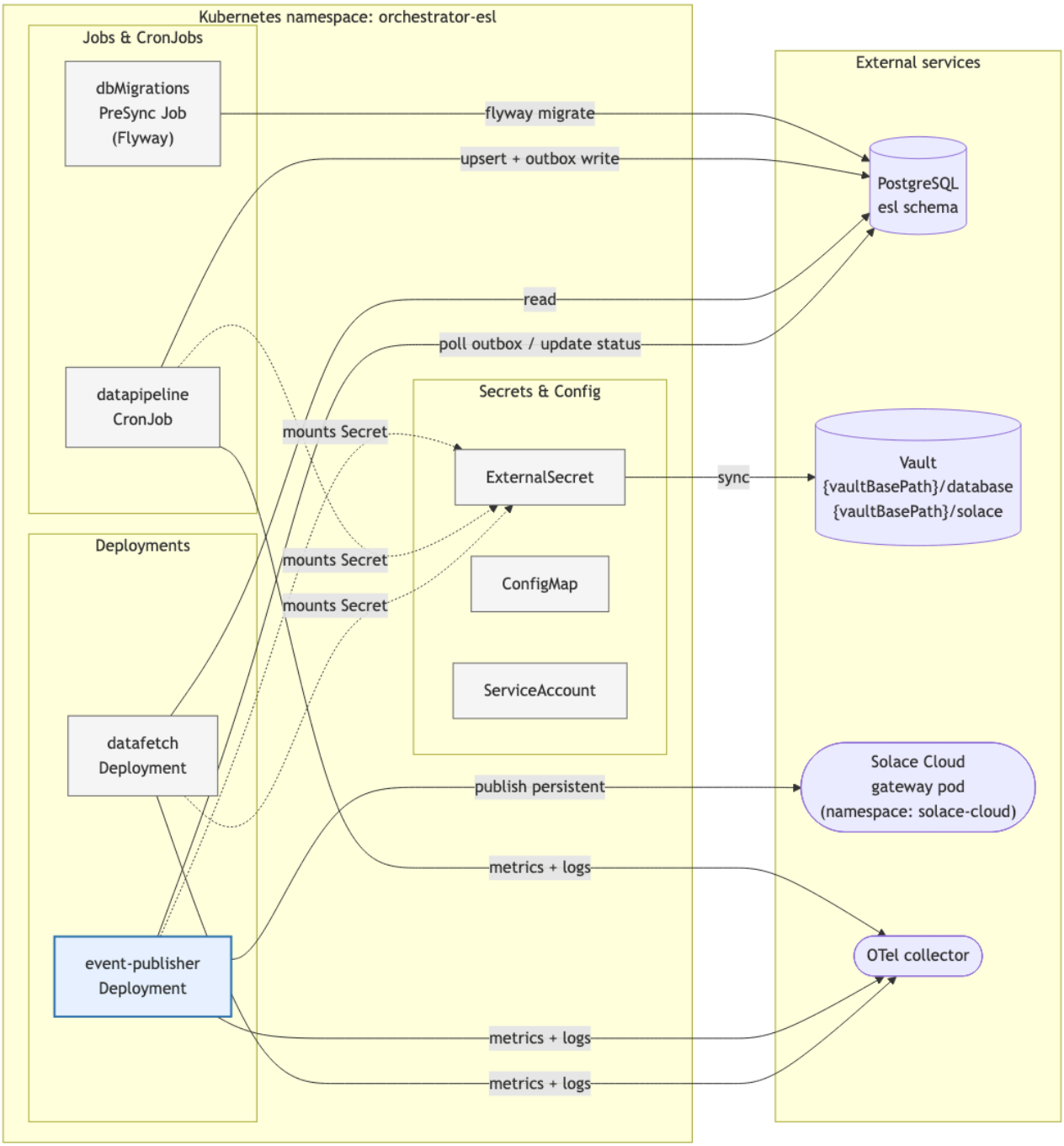


Figure 6: Phase 2 Kubernetes topology

6.7.2. Deployment flow

ArgoCD sync phases (PreSync Flyway Job → workload apply) and the per-env promotion gates (dev → pp → prd). Each environment lands with `cdc.enabled: false`; the CDC flag is flipped separately once the infrastructure is verified.

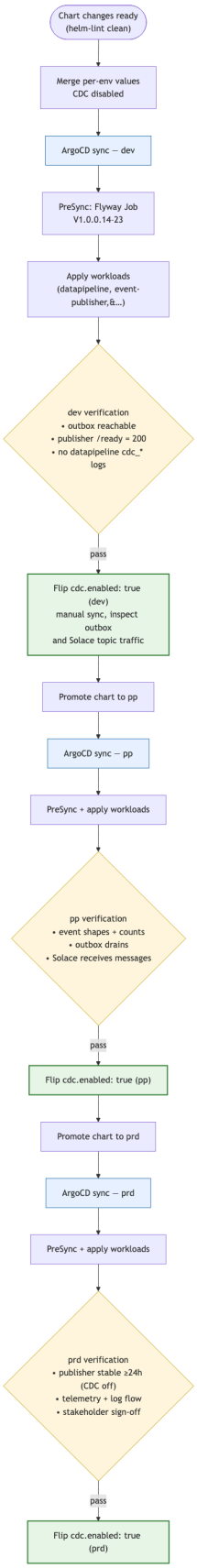


Figure 7: Phase 2 deployment flow

6.8. Configuration Reference

Phase 2 does not introduce new top-level Helm values beyond:

- `dataPipeline.config.cdc.enabled` — bool, default `false`. Controls whether the datapipeline sink writes to `event_outbox`.
- `dataPipeline.suspend` — bool, default unset. When `true`, forces the datapipeline CronJob into a suspended state on top of the active/passive cluster check (`clusterName != activeCluster`). Useful for ad-hoc maintenance pauses on the active cluster without changing `activeCluster`. The override is one-way: setting it to `false` cannot un-suspend a passive cluster — the cluster check still wins.
- `eventPublisher.*` — full event-publisher component configuration. See 05-event-publisher.md for the complete schema.

Per-env overrides live in `clusters/cluster-{env}/values-{env}.yaml`. Security rules (Solace Cloud egress) live in `clusters/cluster-{env}/values-security-{env}.yaml`.

7. Operations & Maintenance

7.1. Overview

Phase 2 operations focus on three areas:

1. Monitoring — outbox health, publisher throughput, Solace connectivity
2. Manual intervention — recovering from failed or stuck events without data loss
3. Capacity planning — when to scale publisher replicas, when to adjust batch or poll settings

7.2. Monitoring

7.2.1. Outbox health (PostgreSQL)

The `event_outbox` table is the primary observability surface. Status counts give an immediate view of system health:

```
SELECT status, COUNT(*)
FROM esl.event_outbox
GROUP BY status;
```

Expected distributions:

Distribution	Meaning
Mostly `DELIVERED`, near-zero `PENDING`	Normal operation — publisher keeps up with producer
Growing `PENDING`	Publisher not running, crashed, or unable to reach Solace
Growing `FAILED`	Transform or publish errors — see runbook

> No initial CREATED/DELETED spike is expected after a CDC enable. The rollout sequence (see 06-deployment.md) populates each environment's database with `cdc.enabled: false`: VLink skips DELETED-row fetches and the sink writes no outbox events during the initial full sync. By the time CDC is flipped on, the entity tables already mirror Vusion's current state — subsequent pre-fetch + diff cycles produce events only on real ongoing changes. If CDC were instead enabled against an empty DB, the first run would emit a CREATED event per active row plus a DELETED event per historical DELETED row VLink returns (≈6,600 outbox entries for a single reference store like `bomdia_pt.009648`: roughly 6,025 product + 631 label DELETED rows). Operators monitoring `event_outbox` post-enable should see volumes proportional to actual change rates, not initial-population magnitudes.

Useful supplementary queries:

```sql
-- Age of oldest pending event
SELECT MIN(occurred_at) AS oldest_pending,
       NOW() - MIN(occurred_at) AS age
FROM esl.event_outbox
WHERE status = 'PENDING';

-- Recent failed events
SELECT id, entity_type, event_type, occurred_at
FROM esl.event_outbox
WHERE status = 'FAILED'
ORDER BY occurred_at DESC
LIMIT 20;

-- Table size (for retention monitoring)
SELECT pg_size_pretty(pg_total_relation_size('esl.event_outbox')) AS total_size,
       COUNT(*) AS rows
FROM esl.event_outbox;
```

7.2.2. Pipeline run lifecycle (`sync_state`)

The orchestrator inserts a `sync_state` row with `sync_status = 'running'` at the start of each run and updates it to `success` / `failed` at the end (see 03-database.md). Retention keeps the last 20 rows per

(pipeline_name, sync_status), so a long success streak no longer evicts the most recent failures. A handful of queries cover the common operational checks:

```
-- Live runs right now (one row per pipeline expected; multiple = concurrency issue)
SELECT pipeline_name, id AS run_id, started_at,
       NOW() - started_at AS age
  FROM esl.sync_state
 WHERE sync_status = 'running'
 ORDER BY started_at;

-- Most recent run per pipeline with outcome
SELECT DISTINCT ON (pipeline_name)
       pipeline_name, sync_status, started_at, finished_at,
       finished_at - started_at AS duration, error_message
  FROM esl.sync_state
 ORDER BY pipeline_name, started_at DESC;

-- Runs swept on startup as orphaned (post-PR-3 indicator of a previous crash)
SELECT pipeline_name, id, started_at, finished_at
  FROM esl.sync_state
 WHERE error_message = 'orphaned by orchestrator restart'
 ORDER BY finished_at DESC LIMIT 20;
```

Alerting note. Time-windowed alerts on sync_status = 'failed' should restrict to finished_at within the window, not started_at — a running row started just before the window has not failed yet, only its eventual terminal update would land inside the window.

7.2.3. Publisher metrics (OpenTelemetry)

The event-publisher exports the following OTel metrics via the shared collector:

- event_publisher.events_processed_total — Counter, attributes entity_type, event_type, status. Investigate when the status=failed rate > 0.
- event_publisher.poll_batch_size — Histogram. Investigate when values are consistently at batch_size (100) — indicates the publisher is lagging.
- event_publisher.poll_cycles_total — Counter. Investigate when it stops incrementing — the publisher is deadlocked or has crashed.
- solace.producer.publish_retries — Counter. Investigate when the rate > 0 — broker latency or connection instability.
- solace.core.connection_drops — Counter. Investigate when any non-zero count appears — broker-side or network issue.

7.2.4. Log filtering (Zap)

The event-publisher uses structured JSON logs. All log entries include `event_id`, `entity_type`, and `status` when relevant. Useful queries:

```
# All warn/error logs in last hour
kubectl logs -l app=orchestrator-esl-eventpublisher --since=1h | \
  jq -c 'select(.level == "warn" or .level == "error")'

# Trace a specific event end-to-end
kubectl logs -l app=orchestrator-esl-eventpublisher --since=24h | \
  jq -c 'select(.event_id == "a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11")'

# Group publish failures by error message
kubectl logs -l app=orchestrator-esl-eventpublisher --since=24h | \
  jq -r 'select(.msg == "publish failed") | .error' | \
  sort | uniq -c | sort -rn
```

7.3. Runbook

7.3.1. Publisher pod stuck in CrashLoopBackOff

Common causes:

1. ExternalSecret has not reconciled — missing or malformed Vault entries
2. Solace credentials invalid — OAuth2 token acquisition or broker authentication fails
3. DB credentials invalid — pool fails to initialize

Diagnosis:

```
kubectl describe pod -l app=orchestrator-esl-eventpublisher | tail -30

# Check ExternalSecret status
kubectl get externalsecret orchestrator-esl-eventpublisher -o yaml | yq '.status'

# Check resulting Secret has all expected keys
kubectl get secret orchestrator-esl-eventpublisher -o json | jq '.data | keys'
```

Expected keys in the resulting Secret:

```
DbHost, DbPort, DbUser, DbPassword, DbName, DbSchema,
SolaceClientId, SolaceClientSecret
```

Non-sensitive Solace config (`host`, `vpn`, `auth_scheme`, `token_endpoint`, `scope`, `topic_prefix`) is sourced from the ConfigMap, not the Secret.

Fix:

- ExternalSecret **`SecretSyncedError`**: verify Vault entries at `{vaultBasePath}/solace` exist and contain `client-id` and `client-secret`. After fixing Vault, force a refresh:

  ```
  kubectl annotate externalsecret orchestrator-esl-eventpublisher force-sync=$(date +%s) --
  overwrite
  ```

- Solace auth fails — broker rejects token: rotate the OAuth2 client secret in the IdP, update `{vaultBasePath}/solace.client-secret` in Vault. If the broker reports issuer/audience/scope mismatch, verify the OAuth profile on the broker still trusts the IdP's issuer and that the requested `scope` (in the env values file under `eventPublisher.config.solace.scope`) matches what the profile expects.
- Solace auth fails — token endpoint unreachable: confirm the IdP's TLS cert is trusted by the pod's system trust store (pod logs will show `x509: certificate signed by unknown authority`) and that egress to the `token_endpoint` host is allowed by the network policy.
- DB auth fails: same procedure, for `{vaultBasePath}/database`.

7.3.2. Outbox `PENDING` backlog is growing

Diagnosis:

```
# Is publisher running?
kubectl get pods -l app=orchestrator-esl-eventpublisher

# Is publisher making poll progress?
kubectl logs -l app=orchestrator-esl-eventpublisher --since=5m --tail=200 | \
  jq -c 'select(.msg | contains("poll cycle"))'

# Is Solace reachable?
kubectl exec -it deploy/orchestrator-esl-eventpublisher -- \
  curl -sf http://localhost:8081/ready
```

Fix:

- Publisher not running: address the CrashLoopBackOff case above.
- Publisher running but `/ready` returns 503: the Solace connection dropped. The publisher auto-reconnects per `reconnect_retries` and `reconnect_wait`. If stuck:

  ```
  kubectl rollout restart deployment/orchestrator-esl-eventpublisher
  ```

7.3.3. Outbox `FAILED` events

Every FAILED event has a reason in the publisher's logs. The three categories:

1. Unsupported entity type — datapipeline wrote an event type the publisher doesn't know (should never happen after Phase 2 ships; indicates a datapipeline bug)
2. Missing key fields — `retail_chain_id` or `store_id` missing from `entity_key`

3. Publish failure — Solace unreachable or confirmation timeout exceeded after internal retries

Recover by resetting to PENDING after fixing the root cause:

```
-- Reset specific failed events (preferred — surgical)
UPDATE esl.event_outbox
SET status = 'PENDING', delivered_at = NULL
WHERE id = ANY('{uuid1,uuid2,uuid3}'::uuid[]);

-- Reset all failed events in a time window
UPDATE esl.event_outbox
SET status = 'PENDING', delivered_at = NULL
WHERE status = 'FAILED'
  AND occurred_at > NOW() - INTERVAL '1 hour';

-- Reset everything FAILED (only if root cause was broker-wide and all should retry)
UPDATE esl.event_outbox
SET status = 'PENDING', delivered_at = NULL
WHERE status = 'FAILED';
```

On the next poll cycle the publisher picks them up.

Consumer deduplication note: delivery is at-least-once. Resetting a FAILED event may cause a duplicate publish if the original actually reached Solace but the status update failed. The `eventId` (outbox UUID) stays the same across retries — consumers dedupe by that field.

7.3.4. Inspect a specific event's contents

```
SELECT
  id,
  event_type,
  entity_type,
  entity_key,
  jsonb_pretty(payload) AS payload,
  status,
  occurred_at,
  delivered_at
FROM esl.event_outbox
WHERE id = 'your-uuid-here';
```

7.3.5. Recover from a falsely-successful store run

A pre–PR-2 run could mark a store as `success` while the sink was silently dropping records (e.g. unique-constraint violations cascading through a pgx batch). The watermark advanced past the failed records, so the next incremental run skipped them. After PR 2 such runs surface as `failed` and the watermark stays put — but historical leaks need a one-time manual recovery.

Symptoms. A store row in `store_sync_state` has `sync_status = 'success'` but the destination table has fewer rows than `*_processed` reports, and `records_with_errors` either contains entries for that run or is suspiciously empty for the same `synced_at`.

Recovery (widen lookback for one run). Roll back the affected store's watermark by the smallest interval that covers the missed records, then trigger a normal run:

```
-- Replace <pipeline>, <retail_chain> and <store> with the affected values.
UPDATE esl.store_sync_state
   SET synced_at = synced_at - INTERVAL '6 hours'
 WHERE pipeline_name = '<pipeline>'
   AND retail_chain_id = '<retail_chain>'
   AND store_id = '<store>'
   AND id = (
     SELECT MAX(id) FROM esl.store_sync_state
      WHERE pipeline_name = '<pipeline>'
        AND retail_chain_id = '<retail_chain>'
        AND store_id = '<store>');
```

Alternative. Bump the orchestrator's `sync.lookback_window` config to a value large enough to cover the gap and run once; revert to the normal window afterwards.

After the recovery run, verify: the store should show `success` with a `*_processed` count consistent with the destination table, and `records_with_errors` should reflect any rows that genuinely failed permanent validation.

7.3.6. Stale `running` row at startup (orphan sweep fired)

On startup, the orchestrator looks for `sync_state` rows belonging to its `pipeline_name` that are still in `running` and flips them to `failed` with `error_message = 'orphaned by orchestrator restart'`. A log line surfaces only when the sweep actually updates rows:

```
{"level":"info","msg":"Swept orphaned running rows","count":1,"pipeline_name":"esl-orchestrator-pp"}
```

This means the previous orchestrator instance for this pipeline died between the start-of-run insert and the end-of-run update — typical causes: pod OOMKilled, node eviction, hard shutdown without SIGTERM, an unhandled panic. The swept row's `started_at` is the timestamp the dead run began; subtract it from the sweep time to estimate how long it was orphaned.

Pipeline-name uniqueness is the safety boundary. The sweep matches strictly on `pipeline_name` and treats every `running` row in scope as dead. If two orchestrators share the same `pipeline_name` and run at the same time, the second one's startup will mark the first one's in-flight row as `failed` while the first is still alive (concurrent orchestrators on a single pipeline-name is documented as out of scope — see 04-datapipeline.md).

Triage:

- Cross-reference the `started_at` of the swept row with pod restart events / kubelet logs to confirm a crash occurred.
- The records the orphaned run was processing remain at the source — the next normal run picks them up via the unchanged `store_sync_state` watermarks.
- A swept row with `started_at` far in the past (hours+) usually indicates the orchestrator was disabled rather than crashed; double-check the deployment was intentional.

If the sweep itself fails (state DB briefly unreachable at startup), a warn log fires and the new running row is still inserted — the next successful startup eventually catches up.

7.3.7. Investigate failed records for a specific run

After PR 3, every row in `records_with_errors` carries `run_id`, `retail_chain_id`, `store_id`, `pipeline_name`, and `entity_type`. Use them to triage a specific failed run:

```
-- All failures for the most recent failed run on a pipeline
WITH latest AS (
  SELECT id FROM esl.sync_state
   WHERE pipeline_name = '<pipeline>'
     AND sync_status = 'failed'
   ORDER BY started_at DESC LIMIT 1
)
SELECT entity_type, retail_chain_id, store_id,
       COUNT(*) AS failures,
       array_agg(DISTINCT pg_error_code) AS pg_codes
  FROM esl.records_with_errors
 WHERE run_id = (SELECT id FROM latest)
 GROUP BY entity_type, retail_chain_id, store_id
 ORDER BY failures DESC;

-- One specific failing record's payload + classification
SELECT data, error_message, pg_error_code, error_type
  FROM esl.records_with_errors
 WHERE run_id = $1 AND record_id = $2;
```

Pre-PR-3 rows. Rows older than the V1.0.0.22 migration carry NULL in all five correlation columns. Dashboards and ad-hoc queries that filter on `run_id` should `WHERE run_id IS NOT NULL` (or accept that pre-migration data is invisible to the new query patterns).

7.3.8. Ack-leak warning in pipeline logs

A log line like the following surfaces from `cmd/eslorchestrator/run.go` after `pipeline.Run` returns:

```
{"level":"warn","msg":"ack leak detected after pipeline run","total_pending":42,"by_store":
{"<store>":42}}
```

This always indicates a code defect — either a pipeline-side drop site that forgot to invoke the OutcomeHandler, or a sink that violated the contract that requires reporting every record before `Close` returns. The corresponding store(s) get classified as `failed: ack leak: N records unaccounted for` in `store_sync_state` so the watermark does not advance.

Triage:

- Cross-reference the affected store IDs with recent connector / sink changes.
- The pipeline's `processRecord` calls the OutcomeHandler on transform/preprocess errors and the sink reports per-record outcomes via `flushBatch`, the `Write()` boundary rejection branch, and `drainOnCancel` on shutdown — any new drop site must do the same.
- Re-run the pipeline once the underlying defect is fixed; the records sit untouched at the source so they will re-emit cleanly.

7.3.9. Drain the outbox urgently (emergency only)

If a backlog is safe to discard (e.g., accumulated test data, intentional rollback), mark events as DELIVERED without publishing:

```
-- DANGER: these events will NOT be sent to Solace.
-- Use only when the backlog must not be published.
UPDATE esl.event_outbox
SET status = 'DELIVERED', delivered_at = NOW()
WHERE status = 'PENDING'
  AND occurred_at < NOW() - INTERVAL '1 day';
```

Preferred alternative: stop the producer (flip `cdc.enabled: false`), let the publisher drain the queue naturally, then re-enable.

7.4. Scaling

7.4.1. Publisher replicas

The publisher uses `SELECT ... FOR UPDATE SKIP LOCKED`, so multiple replicas can run concurrently without coordination. Each replica claims distinct rows.

Single replica (default):

- Per-entity ordering is preserved — events for the same entity are always delivered in outbox order
- Simpler to monitor (single log stream, single publisher identity in metrics)
- Sufficient for typical ESL workloads (the theoretical ceiling at `pollInterval: 1s, batchSize: 100` is 10,000 events/sec, far above observed rates)

Multiple replicas:

- Scales throughput up to the broker's accepted rate
- Breaks per-entity ordering — two updates to the same entity may be claimed by different replicas and arrive at Solace out of order
- Required only if single-replica throughput saturates or if high-availability during rolling restart matters
- Increase `replicaCount` in `values-{env}.yaml` under `eventPublisher`

7.4.2. Batch size and poll interval

Setting	Default	Increase when	Decrease when
`publisher.batchSize`	100	Outbox backlog grows, events produced > consumed	Large batches cause long transaction locks; memory pressure
`publisher.pollInterval`	1s	Events are sparse, want to reduce DB load	Latency-sensitive delivery needed

Changes require a Helm values edit and redeploy — no DB migration involved.

7.4.3. Data pipeline CDC overhead

As documented in 04-datapipeline.md, CDC adds approximately 30–100% overhead per batch depending on the change ratio. Mitigations if runtime becomes a problem:

1. Profile which entity types drive the most events using the `sink.cdc.events` metric grouped by `event_type`
2. Lower `sink.postgres.batch_size` (currently 500) if per-batch transaction time is too long
3. For one-off heavy loads (full re-sync), temporarily disable CDC (`cdc.enabled: false`), run the sync, then re-enable

7.5. Outbox Retention

Migration V1.0.0.18 (event_outbox_retention) handles periodic cleanup:

- DELIVERED rows older than the retention window (default: 7 days) are purged
- FAILED rows are never purged automatically — manual intervention is required

Monitor table size:

```
SELECT pg_size_pretty(pg_total_relation_size('esl.event_outbox')) AS total_size;

-- Count of rows eligible for purge
SELECT COUNT(*)
FROM esl.event_outbox
WHERE status = 'DELIVERED'
```

```
  AND delivered_at < NOW() - INTERVAL '7 days';

-- Count of unresolved FAILED rows (investigate if this grows)
SELECT COUNT(*) FROM esl.event_outbox WHERE status = 'FAILED';
```

7.6. Graceful Shutdown

The event-publisher handles SIGTERM cleanly:

1. Kubernetes sends SIGTERM (e.g., during rolling deployment)
2. Publisher stops accepting new poll cycles, finishes any in-flight batch
3. In-flight transaction commits — rows become DELIVERED or FAILED; no rows left in intermediate state
4. Solace client disconnects cleanly
5. Pod exits

If `shutdownTimeout` (default `30s`) is exceeded, Kubernetes sends SIGKILL. Consequences:

- The in-flight transaction rolls back; rows revert to PENDING
- The next pod picks them up on startup
- No data loss (at-least-once)
- Possible duplicate publishes if the pre-rollback batch had partially delivered to Solace (consumers dedupe by `eventId`)

7.7. Active/Passive Failover

> Note: this is a temporary mechanism. The active/passive setup and the manual failover procedure described below are interim measures adopted to enable multi-cluster operation within Phase 2 timelines. Future phases of the project will replace this with a more robust approach to multi-cluster scheduling — likely a DB-backed lease for automatic leader election — removing the need for coordinated `activeCluster` values flips. The values-flip mechanism documented here is not the long-term design.

The ESL Orchestrator is deployed to two OpenShift clusters per environment in an active/passive pattern for the components that publish externally — the datapipeline CronJob and the event-publisher Deployment. One cluster is designated active. There the CronJob runs on schedule and the Deployment runs at its configured replica count. The other cluster renders the same chart but with `spec.suspend: true` on the CronJob and `spec.replicas: 0` on the Deployment, so both resources exist but neither fires nor runs pods. The `datafetch` REST API stays active/active across both clusters; only the two publishing components are gated. Gating event-publisher avoids duplicate publishes to Solace from two clusters running in parallel.

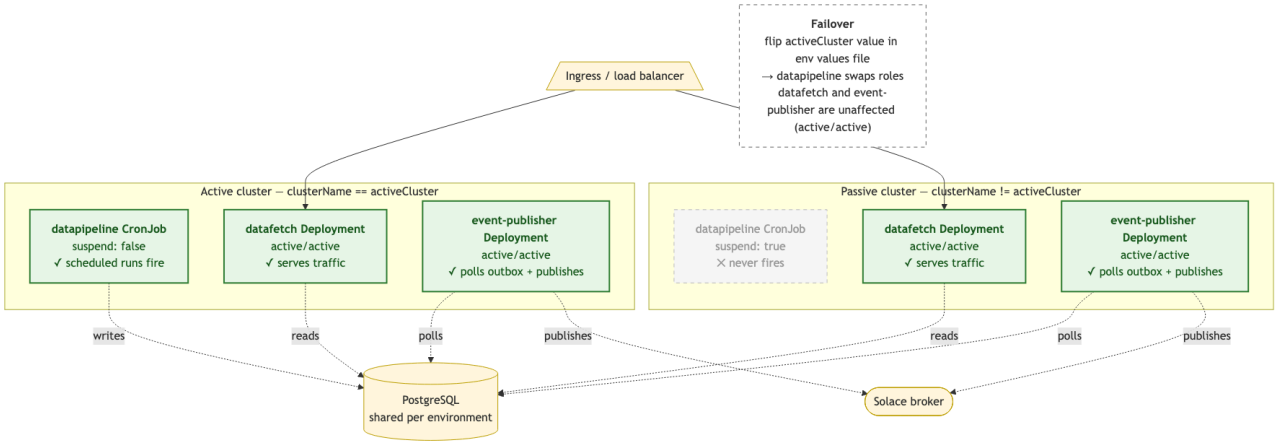


Figure 8: Active/Passive cluster topology

7.7.1. Mechanism

Two values drive the decision at Helm render time:

Value	Source	Per cluster?
`clusterName`	Injected by ArgoCD via `helm.parameters` in each per-cluster Application	Yes — each cluster's Argo passes its own identity
`activeCluster`	Top-level value in `clusters/cluster-{env}/values-{env}.yaml`	No — shared across both clusters in an environment

The datapipeline CronJob template renders:

```
suspend: {{ or $dp.suspend (ne .Values.clusterName .Values.activeCluster) }}
```

On the active cluster, `clusterName == activeCluster` → `suspend: false`. On the passive cluster, `clusterName != activeCluster` → `suspend: true`.

The event-publisher Deployment template renders:

```
replicas: {{ if ne .Values.clusterName .Values.activeCluster }}0{{ else }}{{ $ep.replicaCount | default 1 }}{{ end }}
```

On the active cluster, replicas resolves to the configured `eventPublisher.replicaCount` (default `1`). On the passive cluster it resolves to `0`, so the Deployment object exists but has no pods.

The `dataPipeline.suspend` override (see o6-deployment.md) is an optional boolean that force-suspends the CronJob on top of the cluster check — useful for ad-hoc maintenance pauses on the active cluster without touching `activeCluster`. It is one-way: setting it to `false` cannot un-suspend the passive cluster. There is no equivalent override for event-publisher; to pause it on the active cluster, scale the Deployment manually or set `eventPublisher.replicaCount: 0`.

Cluster identities per environment:

Environment	Active	Passive
pp	`oshift-pp-rba1`	`oshift-pp-mts1`
prd	`oshift-prd-rba1`	`oshift-prd-mts1`

7.7.2. Identifying the active cluster as unhealthy

Signals that warrant a failover:

- Datapipeline has not had a successful run on the active cluster across a full schedule window (prd schedule: `00 12 * * *`).
- OpenShift cluster-level outage affecting the active cluster (node failures, network partition, control plane degraded).
- ArgoCD unable to sync the ESL Application on the active cluster.

Quick checks:

```
# Latest datapipeline Job status on the active cluster
kubectl --context <active-cluster> get jobs -n instore-esl-orchestrator-prd \
  -l app=orchestrator-esl-datapipeline --sort-by=.status.startTime | tail -5

# Latest esl.sync_state row
psql -c "SELECT client, sync_status, sync_end_time FROM esl.sync_state
         WHERE client = 'esl-orchestrator-prd'
         ORDER BY sync_end_time DESC LIMIT 5;"
```

7.7.3. Performing a failover

1. Edit the env's values file in the `sonaemc-instore/lac1041-instoreorchestrator_esl` repo and flip `activeCluster` to the passive cluster's name. For a prd failover from `rba1` to `mts1`:

   ```
   # clusters/cluster-production/values-prd.yaml
   activeCluster: "oshift-prd-mts1"
   ```

2. Commit and push. The same values file is consumed by both per-cluster Argo Applications, so a single edit reaches both clusters.

3. Wait for Argo to sync on both clusters (or force a sync from the Argo UI).

7.7.4. Post-failover verification

Confirm the CronJob `suspend` flag flipped and the event-publisher Deployment scaled on both clusters:

```
# New active cluster — expect CronJob "false" and Deployment replicas >= 1
kubectl --context <new-active> get cronjob -n instore-esl-orchestrator-prd \
  orchestrator-esl-datapipeline -o jsonpath='{.spec.suspend}'
```

```
kubectl --context <new-active> get deploy -n instore-esl-orchestrator-prd \
  orchestrator-esl-eventpublisher -o jsonpath='{.spec.replicas}'

# Old active (now passive) — expect CronJob "true" and Deployment replicas 0
kubectl --context <old-active> get cronjob -n instore-esl-orchestrator-prd \
  orchestrator-esl-datapipeline -o jsonpath='{.spec.suspend}'
kubectl --context <old-active> get deploy -n instore-esl-orchestrator-prd \
  orchestrator-esl-eventpublisher -o jsonpath='{.spec.replicas}'
```

After the next schedule tick:

- A new Job is created on the new active cluster; no Job is created on the old active.
- `esl.sync_state` shows one new row for `esl-orchestrator-prd` with the expected `sync_end_time`.
- Event-publisher pods are running on the new active cluster and absent on the old active; new outbox rows are drained by the new active's pods.

7.7.5. Rollback

Edit the same values file and flip `activeCluster` back to the original cluster. Verification steps are identical — `suspend: false` and event-publisher replicas >= 1 should reappear on the original active cluster, and `suspend: true` with replicas `0` on the other.

7.8. Disaster Recovery

7.8.1. Outbox table corrupted or lost

The outbox is not a system of record — entity data in `stores`, `products`, `labels`, `access_points` is. Events can be reconstructed by forcing a full re-sync:

1. Stop the datapipeline CronJob (suspend it)
2. Reset `store_sync_state` to an earlier timestamp to force re-sync of all changed-since data
3. Re-enable the CronJob
4. Next run produces fresh events into the rebuilt outbox
5. Publisher delivers them

Caveat: re-synced events report as UPDATED (not CREATED), because the data is already in place. Consumers that distinguish the two must account for this.

7.8.2. Solace Cloud extended outage

The publisher marks failed publishes as FAILED. Short outages recover automatically via the publisher's internal retries; longer outages cause the backlog to grow.

For prolonged outages (hours), monitor:

- `event_outbox` row count and disk usage

- `/ready` endpoint status on the publisher pods

When Solace recovers, reset FAILED events to PENDING (see runbook) and allow the publisher to drain the backlog.

7.8.3. Publisher unable to keep up

If the publisher consistently cannot drain the outbox (publish rate < produce rate):

1. Scale `replicaCount` up (accepting the loss of per-entity ordering)
2. Investigate Solace broker-side limits (queue depth, publish rate throttles)
3. Consider partitioning the outbox by entity_type at the consumer side (out of scope for Phase 2, revisit in Phase 3)